

Προηγμένες Δομές Δεδομένων στην Υπολογιστική Γεωμετρία

Αντρέας Παναγή*

Ερευνητική Μονάδα Αρχιμήδης του Ερευνητικού Κέντρου Αθηνά

*Υπό την επίβλεψη του Δρ. Κώστα Τσακαλίδη

Περιεχόμενα

1	Εισαγωγή	4
1.1	Αναφορά πρακτικής εργασίας	4
1.2	Τομέας έρευνας και στόχοι του project	4
2	Μοντέλα Υπολογισμού και Μέτρα Πολυπλοκότητας	6
2.1	RAM και Παραλλαγές	6
2.2	Cell-probe μοντέλο	7
2.3	Pointer machine και comparison model	7
2.4	External-memory / I/O model	7
2.5	Μέτρα Πολυπλοκότητας (Complexity Measures)	8
3	Εισαγωγή στις Δομές Δεδομένων	10
3.1	Στατικές και Δυναμικές Δομές Δεδομένων	10
3.2	Διαχρονικότητα (Persistence)	10
3.3	Succinct, compact and implicit δομές	11
4	Δενδρικές Δομές Δεδομένων	12
4.1	Χάρτης Δενδρικών Δομών	12
4.2	Aggregation trees	13
4.2.1	Tournament trees	13
4.3	Search trees	14
4.3.1	Multi-way Trees	14
4.3.2	(a,b)-trees	15
4.3.3	Binary Search Trees: AVL και Red-Black trees	16
4.3.4	Treaps	16
4.4	String trees	17
4.4.1	Tries	17
4.5	Integer Trees	18
4.5.1	van Emde Boas Tree	18
4.6	Fusion Trees	20
5	Computational-Geometry Trees	22
5.1	Interval tree	22
5.2	Segment tree	23
5.3	Kd-tree	24
5.4	Range Trees	26
6	Επιλεγμένες Προηγμένες Δομές Δεδομένων	28
6.1	Priority Queue with Attrition	28
6.2	Catenable Priority Queue with Attrition	29
6.3	Q-Heap	30
7	External Memory και Buffering Framework	31
7.1	B-trees	31
7.2	Παραλλαγές των B-Trees	32
7.3	Buffer tree	33
7.4	Buffering Framework	34
7.4.1	Μετατροπή Red Black Tree σε Buffered Red Black Tree: Απόδειξη	34
7.5	Έξτρα: Ταξινόμηση στην εξωτερική μνήμη	36

8	Micro–Macro Τεχνική: Μια Ανασκόπηση	37
8.1	Rank-space reduction	37
8.1.1	Δυναμικό Σύμπαν: List Labeling	37
8.2	Ζέσταμα: Square Root Decomposition	38
8.3	Van Emde Boas Transformation	39
8.4	Micro - Macro Τεχνική	40
9	Προγραμματιστική Υλοποίηση	41
10	Ανασκόπηση	41

1 Εισαγωγή

Αυτή η ενότητα παρέχει μια συνοπτική επισκόπηση της αναφοράς. Περιγράφει το πλαίσιο της πρακτικής άσκησης, τη χρονική διάρκεια, τους κύριους στόχους του έργου και τη δομή των επόμενων κεφαλαίων που υποστηρίζουν την τεκμηρίωση και την αξιολόγησή του.

1.1 Αναφορά πρακτικής εργασίας

Η παρούσα αναφορά συνοψίζει το ερευνητικό έργο που εκπονήθηκε κατά την πρακτική περίοδο, από τον Ιούνιο 2025 έως το τέλος Νοεμβρίου 2025, υπό την επίβλεψη του Δρ. Κ. Τσακαλίδη. Το έργο πραγματοποιήθηκε στην Ερευνητική Μονάδα Αρχιμήδης του Ερευνητικού Κέντρου Αθηνά και επικεντρώθηκε στη σχεδίαση και ανάλυση δομών δεδομένων για προβλήματα υπολογιστικής γεωμετρίας. Επιπλέον, μέρος της εργασίας περιλάμβανε την υλοποίηση και αξιολόγηση παραλλαγών της *Priority Queue with Attrition* (PQA) σε Python και C++, τόσο σε *amortized* όσο και σε *worst-case* παραλλαγές, καθώς και υλοποιήσεις με *forward* και *backward cleaning schemes*.

1.2 Τομέας έρευνας και στόχοι του project

Κεντρικό αντικείμενο της εργασίας ήταν η μελέτη και η εφαρμογή της τεχνικής *Micro-Macro*, όπως περιγράφεται στη δημοσίευση «Dynamic Orthogonal Range Searching on the RAM, Revisited» [CT17]. Στόχος ήταν η εις βάθος κατανόηση της μεθοδολογίας *Micro-Macro*, η διερεύνηση εναλλακτικών επιλογών υλοποίησης και η αξιολόγηση της αποτελεσματικότητάς της σε προβλήματα *range searching*.

Ο γενικός τομέας έρευνας είναι οι προηγμένες δομές δεδομένων στην υπολογιστική γεωμετρία. Οι κύριοι επιστημονικοί και τεχνικοί στόχοι του project ήταν οι εξής:

- να καταγράψω και να αναλύσω το θεωρητικό υπόβαθρο και τα βασικά μοντέλα υπολογισμού (Word-RAM, pointer machine, external-memory, cell-probe) που καθορίζουν τα άνω και κάτω φράγματα απόδοσης δομών που εμφανίζονται σε γεωμετρικά προβλήματα,
- να μελετήσω και να τεκμηριώσω κεντρικές τεχνικές σχεδίασης (rank-space reduction, list labeling, square-root decomposition, fractional cascading, persistence, buffering) που χρησιμοποιούνται ως εργαλεία για την κατασκευή CPU- και I/O-αποδοτικών γεωμετρικών δομών,
- να οργανώσω και να παρουσιάσω βασικές κατηγορίες δενδρικών δομών και τις παραλλαγές τους (aggregation/search/string/integer trees) ως πλαίσιο για τις γεωμετρικές δομές που ακολουθούν,
- να μελετήσω δενδρικές δομές της υπολογιστικής γεωμετρίας (interval trees, segment trees, kd-trees, range trees) και τη σχέση τους με τις λειτουργίες αναζήτησης και ενημέρωσης σε δυναμικές ρυθμίσεις,
- να εξοικειωθώ με το εξωτερικής μνήμης πλαίσιο και τις τεχνικές buffering (B-trees, παραλλαγές, Buffer Tree, γενικό buffering framework) ως υποδομή για I/O-αποδοτικές δομές,
- να μελετήσω και να συνοψίσω την τεχνική *Micro-Macro* (Chan-Tsakalidis) και τα προπαρασκευαστικά της βήματα, με έμφαση στον μετασχηματισμό van Emde Boas και στον ρόλο του στη σύνδεση *micro-* και *macro-*δομών,

- να υλοποιήσω επιλεγμένες δομές (PQA/CPQA και βοηθητικές υπορουτίνες) ώστε να αποτυπώσω πρακτικά ζητήματα υλοποίησης και ενδεικτικές επιδόσεις.

Στις επόμενες ενότητες παρουσιάζονται τα μοντέλα υπολογισμού και τα βασικά μέτρα πολυπλοκότητας, και έπειτα μια σύντομη εισαγωγή σε κεντρικές έννοιες δομών δεδομένων, όπως οι στατικές και δυναμικές δομές, η διαχρονικότητα (persistence) και οι succinct/compact/implicit αναπαραστάσεις. Ακολουθεί χαρτογράφηση και ανάλυση βασικών δένδρικών σχημάτων, με ενότητες για aggregation trees, search trees, string trees και integer trees. Στη συνέχεια, εξετάζονται επιλεγμένες δένδρικές δομές της υπολογιστικής γεωμετρίας, με επίκεντρο τα interval trees, segment trees, kd-trees και range trees, καθώς και ορισμένες «δομικές» προηγμένες δομές που χρησιμοποιούνται ως building blocks (Priority Queue with Attrition, Catenable PQA, Q-Heap). Η ενότητα για εξωτερική μνήμη συγκεντρώνει τα βασικά εργαλεία I/O-αποδοτικού σχεδιασμού, ξεκινώντας από τα B-trees, τις παραλλαγές τους και το Buffer Tree, και ολοκληρώνεται με μια σύντομη παρουσίαση της ταξινόμησης στο I/O μοντέλο ως δεμελιώδους υπορουτίνας. Τέλος, παρουσιάζεται η τεχνική Micro–Macro με τα απαραίτητα προπαρασκευαστικά βήματα (rank-space reduction, list labeling, square-root decomposition), τον μετασχηματισμό van Emde Boas και τη συνολική εικόνα της μεθοδολογίας. Η ενότητα «Υλοποίηση» συνοψίζει τις βασικές επιλογές της εργαστηριακής εφαρμογής και η τελική «Ανασκόπηση» συγκεντρώνει τα κύρια τεχνικά και μαθησιακά συμπεράσματα.

2 Μοντέλα Υπολογισμού και Μέτρα Πολυπλοκότητας

Τα μοντέλα υπολογισμού καθορίζουν το σύνολο των επιτρεπτών λειτουργιών και την κοστολόγησή τους, επομένως επηρεάζουν άμεσα τις θεωρητικές εγγυήσεις χρόνου και χώρου των δομών δεδομένων. Στην παρούσα εργασία εξετάζουμε ειδικά το μοντέλο Word-RAM, το pointer-machine / comparison model, το external-memory / I/O model και το cell-probe model. Οι δομές δεδομένων που θα αναλύσουμε στα μετέπειτα κεφάλαια θα αντιστοιχούν σε (τουλάχιστον) ένα από τα παραπάνω μοντέλα.

Επιπλέον, θα ορίζουμε τα μέτρα πολυπλοκότητας που θα χρησιμοποιούνται στα επόμενα κεφάλαια: worst-case χρόνο, expected (αναμενόμενο) χρόνο και επιμερισμένο (amortized) χρόνο. Τα προαναφερθέντα μέτρα χρησιμοποιούνται, κατά κανόνα, στην ανάλυση των δομών δεδομένων και στη σχεδίαση σύνθετων δομών που εμφανίζονται στη βιβλιογραφία, ιδίως στις δημοσιεύσεις με τις οποίες ασχολήθηκε η παρούσα πρακτική.

2.1 RAM και Παραλλαγές

Random Access Machine (RAM) μοντέλο. Στο μοντέλο RAM η μνήμη αποτελείται από κελιά (cells) και η βασική μέτρηση κόστους είναι ο αριθμός των προσπελάσεων μνήμης (αναγνώσεις/εγγραφές) και των στοιχειωδών πράξεων. Τυπικά, κάθε ανάγνωση ή εγγραφή σε κελί κοστίζει $O(1)$, όπως και οι βασικές αριθμητικές και συγκριτικές πράξεις (π.χ. $+$, $-$, $\times$, $/$, $\%$, καθώς και $<$, $>$, $=$). Στο «καθαρά» θεωρητικό RAM η μνήμη θεωρείται απείρως διαθέσιμη (unbounded abstraction). Το RAM χρησιμοποιείται ως αφηρημένο πλαίσιο για τη διατύπωση εγγυήσεων χρόνου και χώρου. [CLR+09]

Word-RAM μοντέλο. Στο word-RAM κάθε κελί αποθηκεύει μία λέξη μήκους w bits, δηλαδή μία τιμή στο $\{0, \dots, 2^w - 1\}$. Οι βασικές πράξεις πάνω σε λέξεις (αριθμητικές, συγκρίσεις) θεωρούνται $O(1)$, ενώ επιτρέπονται επίσης $O(1)$ bitwise πράξεις και μετατοπίσεις (AND, OR, XOR, left/right shifts). Μετράμε τον χώρο σε λέξεις: «χώρος S » σημαίνει S λέξεις μνήμης. Συνήθως υιοθετείται η *transdichotomous* παραδοχή $w \geq \log n$, ώστε μία λέξη να αρκεί για δείκτες/μετρητές μεγέθους n . [FLN+12; FW93]

Σημείωση. Πολλές βελτιώσεις για *integer-based* δομές εκμεταλλεύονται word-level parallelism (bitwise πράξεις, shifts, και το μέγεθος w) για να επιτύχουν χρόνους καλύτερους από το καθαρά comparison-based πλαίσιο. Για παράδειγμα, δομές όπως τα *fusion trees* δίνουν χρόνους αναζήτησης $O(\log n / \log w)$, που υπό τυπικές παραδοχές οδηγούν σε $O(\log n / \log \log n)$. Επιπλέον, υπάρχουν αλγόριθμοι ταξινόμησης ακεραίων που ξεπερνούν το $O(n \log n)$, όπως $O(n \log n / \log \log n)$ (μέσω fusion trees) και ακόμη ταχύτερα όρια υπό πρόσθετες υποθέσεις εύρους/μοντέλου. [FW93; Han04]

Real-RAM μοντέλο. Το Real-RAM εισήχθη από τον Shamos και αποτελεί παραλλαγή του RAM όπου τα κελιά μνήμης μπορούν να αποθηκεύουν ακριβείς πραγματικούς αριθμούς. Στο Real-RAM οι βασικές αριθμητικές πράξεις (πρόσθεση, αφαίρεση, πολλαπλασιασμός, διαίρεση) και οι συγκρίσεις μεταξύ πραγματικών θεωρούνται ότι εκτελούνται σε $O(1)$ χρόνο, χωρίς να λαμβάνεται υπόψη το κόστος αναπαράστασης (π.χ. πλήθος bits). Χρησιμοποιείται ευρέως στην υπολογιστική γεωμετρία ως μοντέλο που απλοποιεί τη διαχείριση πραγματικών συντεταγμένων. [Sha78]

Σημείωση. Σε πολλές διατυπώσεις του Real-RAM αποφεύγονται πράξεις που αυξάνουν υπερβολικά τη θεωρητική ισχύ του μοντέλου (π.χ. $\lfloor \cdot \rfloor$, modulus σε σταθερό κόστος), διότι η μίξη «ακρίβειας πραγματικών» με τέτοιες ακέραιες λειτουργίες μπορεί να οδηγήσει σε μη

ρεαλιστική υπολογιστική ισχύ (π.χ. επίλυση PSPACE προβλημάτων σε πολυωνυμικό χρόνο υπό unit-cost παραδοχές). [EHM21; Ann03]

2.2 Cell-probe μοντέλο

Ορισμός. Το *cell-probe model* είναι αφαιρετική παραλλαγή του word-RAM, στην οποία το μοναδικό κοστολογούμενο μέτρο είναι ο αριθμός των προσπελάσεων σε κελιά μνήμης (cell probes). Η μνήμη αποτελείται από S κελιά, κάθε ένα με χωρητικότητα w bits, και οποιοσδήποτε τοπικός υπολογισμός μεταξύ προσπελάσεων θεωρείται «δωρεάν». [Yao81; FS89; Mil00]

Το μοντέλο αυτό είναι κατάλληλο για απόδειξη κάτω ορίων και ανάλυση trade-offs χώρου–χρόνου· αποτελέσματα σε αυτό το μοντέλο μεταφέρονται συχνά στο word-RAM μέσω της παραμέτρου w . [Mil00; PD06]

2.3 Pointer machine και comparison model

Pointer machine. Το *pointer machine* είναι θεωρητικό μοντέλο υπολογισμού όπου η μνήμη οργανώνεται ως γράφημα από κόμβους σταθερού μεγέθους, συνδεδεμένους με δείκτες. Κάθε κόμβος έχει πεπερασμένο αριθμό πεδίων (data fields) και δεικτών (pointers), και η πρόσβαση στα δεδομένα γίνεται αποκλειστικά μέσω «κυνηγητού δεικτών» (pointer chasing) από μια ή περισσότερες γνωστές «ρίζες» (root pointers). [Bla04; Dem+12]

Οι επιτρεπτές στοιχειώδεις πράξεις στο pointer machine περιλαμβάνουν:

- πρόσβαση σε έναν κόμβο μέσω υπάρχοντος δείκτη,
- ανάγνωση ή εγγραφή σε κάποιο πεδίο δεδομένων ενός κόμβου,
- ανάγνωση ή τροποποίηση ενός δείκτη,
- απλοί ελέγχοι ισότητας/ανισότητας επί των πεδίων.

Κάθε τέτοια πράξη κοστίζει $O(1)$ χρόνο. Η ισχύς του μοντέλου εστιάζει στη δομή του γραφήματος δεικτών. [CR96]

Comparison model. Το *comparison model* είναι συμπληρωματικό πλαίσιο ανάλυσης στο οποίο η πολυπλοκότητα ενός αλγορίθμου μετράται ως προς τον αριθμό συγκρίσεων μεταξύ κλειδίων. Οι επιτρεπτές ατομικές πράξεις θεωρούνται συγκρίσεις της μορφής $x < y$, $x > y$, $x = y$ μεταξύ στοιχείων της εισόδου. Η συμπεριφορά του αλγορίθμου μπορεί να αναπαρασταθεί ως δέντρο αποφάσεων (decision tree) όπου κάθε εσωτερικός κόμβος αντιστοιχεί σε μία σύγκριση και κάθε φύλλο σε μία πιθανή έκβαση/διάταξη. [Dem+09; Knu98] Σε αυτό το μοντέλο προκύπτει, για παράδειγμα, το κλασικό κάτω όριο $\Omega(n \log n)$ συγκρίσεων για την ταξινόμηση n στοιχείων με αλγορίθμους βασισμένους αποκλειστικά σε συγκρίσεις. [CLR+09; Dem+09; Knu98]

2.4 External-memory / I/O model

Ορισμός. Το μοντέλο *εξωτερικής μνήμης* (external memory model), ή αλλιώς *I/O-model*, εισήχθη από τους Aggarwal και Vitter το 1988. Σκοπός του μοντέλου είναι η αποτύπωση του κόστους πρόσβασης σε ιεραρχίες μνήμης όπου το κυρίαρχο κόστος είναι η μεταφορά μπλοκ δεδομένων μεταξύ αργής (εξωτερικής) και γρήγορης (εσωτερικής) μνήμης. [AV88; Vit01]

Το μοντέλο περιγράφεται με τις παραμέτρους:

- M : μέγεθος εσωτερικής μνήμης (σε στοιχεία/λέξεις),

- B : μέγεθος μπλοκ (block) μεταφοράς μεταξύ εξωτερικής και εσωτερικής μνήμης,
- N : μέγεθος της εισόδου (αριθμός στοιχείων).

Συχνά υποθέτουμε επίσης ότι κάθε στοιχείο/λέξη έχει σταθερό μήκος $\Theta(w)$ bits, για κάποια σταθερή παράμετρο w , ώστε το μοντέλο να είναι συνεπές με το word-RAM ως προς το μέγεθος των λέξεων. [AV88; Vit01]

Η βασική υπόθεση είναι ότι τα δεδομένα αποθηκεύονται στη δευτερεύουσα μνήμη σε συνεχόμενα μπλοκ μεγέθους B , και η βασική μονάδα κόστους είναι η μεταφορά (read/write) ενός τέτοιου μπλοκ μεταξύ εξωτερικής και εσωτερικής μνήμης. Κάθε μεταφορά μπλοκ μετρά ως μία I/O πράξη ($O(1)$). Η εσωτερική επεξεργασία (υπολογισμός μέσα στα M στοιχεία) θεωρείται πολύ φθηνότερη από τις I/O πράξεις και δεν κοστολογείται σε I/O-ανάλυση. [AV88; Vit01]

Παραλλαγές.

- *Cache-aware* δομές: γνωρίζουν τα M, B και προσαρμόζουν ανάλογα παραμέτρους (π.χ. βαθμός διακλάδωσης σε B-tree). [BM72; Vit01]
- *Cache-oblivious* δομές: δεν γνωρίζουν τα M, B , αλλά (υπό την tall-cache υπόθεση) πετυχαίνουν ασυμπτωτικά βέλτιστο I/O-κόστος σε κάθε επίπεδο ιεραρχίας. [FLP+12; BDF05; Pro99]

2.5 Μέτρα Πολυπλοκότητας (Complexity Measures)

Στην ανάλυση των δομών που παρουσιάζονται σε αυτή την αναφορά χρησιμοποιούμε δύο βασικά μέτρα πολυπλοκότητας: *χειρότερη περίπτωση* (*worst-case*) και *αποσβεστικό κόστος* (*amortized*). Για λόγους πληρότητας αναφέρουμε και την ανάλυση *μέσης περίπτωσης* (*average-case*) καθώς και την ανάλυση *αναμενόμενης περίπτωσης* (*expected*). Παρακάτω δίνονται σύντομοι ορισμοί και χρήσιμες παρατηρήσεις για το καθένα. [CLR+09; Knu98]

Χειρότερη περίπτωση (Worst-case). Ο χρόνος (ή το κόστος) μίας πράξης στη χειρότερη περίπτωση ορίζεται ως το μέγιστο κόστος που μπορεί να απαιτηθεί για αυτή τη πράξη πάνω σε οποιαδήποτε έγκυρη είσοδο του κατάλληλου μεγέθους. Αν $T(I)$ είναι ο χρόνος για είσοδο I μεγέθους n , τότε

$$T_{\text{wc}}(n) = \max_{|I|=n} T(I).$$

Χρησιμοποιείται όταν απαιτούμε εγγυημένο (ντετερμινιστικό) ανώτατο όριο.

Μέση περίπτωση (Average-case). Η μέση περίπτωση αναφέρεται στο μέσο κόστος μιας πράξης υπό ορισμένη κατανομή εισόδων D . Ορίζουμε

$$T_{\text{avg}}(n) = \mathbb{E}_{I \sim D}[T(I)],$$

και η ορθότητα/σημασία της εξαρτάται κρίσιμα από την επιλογή της κατανομής D . Όταν αναφέρεται «average-case», πρέπει να διευκρινίζεται ποια κατανομή εισόδων θεωρείται. [Knu98]

Αναμενόμενη / τυχαιοποιημένη περίπτωση (Expected / Randomized). Σε τυχαιοποιημένους αλγορίθμους η αναμενόμενη (expected) πολυπλοκότητα είναι η προσδοκία του χρόνου ως προς τα τυχαία νομίσματα του αλγορίθμου, για μια δεδομένη είσοδο I :

$$T_{\text{exp}}(I) = \mathbb{E}_{\text{random}}[T(I, \text{coins})],$$

και συχνά αναφέρουμε $T_{\text{exp}}(n) = \max_{|I|=n} T_{\text{exp}}(I)$. Σημειώστε ότι εδώ η προσδοκία δεν προέρχεται από κατανομή εισόδων αλλά από την εσωτερική στοχαστικότητα του αλγορίθμου. [MR95]

Επιμερισμένο κόστος (Amortized). Το επιμερισμένο κόστος ενός τύπου πράξεων ορίζεται ως το μέσο κόστος ανά πράξη σε μια ακολουθία m πράξεων (συνήθως για τη χειρότερη ακολουθία). Αν το συνολικό πραγματικό κόστος για μια ακολουθία A μήκους m είναι $T_{\text{total}}(A)$, τότε το επιμερισμένο κόστος είναι

$$\bar{T}(m) = \max_{|A|=m} \frac{T_{\text{total}}(A)}{m}.$$

Τρόποι ανάλυσης περιλαμβάνουν την *aggregate method*, την *accounting method* και την *potential method*. [Tar85; CLR+09]

3 Εισαγωγή στις Δομές Δεδομένων

Στις παρακάτω υπό-ενότητες παρέχονται ορισμοί και επεξηγήσεις δεμελιωδών εννοιών και τεχνικών που χρησιμοποιούνται στην κατασκευή και ανάλυση προηγμένων δομών δεδομένων. Η κατανόηση αυτών των εννοιών είναι απαραίτητη για την ερμηνεία των αλγοριθμικών εγγυήσεων και για την εφαρμογή τους σε υπολογιστικά προβλήματα, όπως προβλήματα υπολογιστικής γεωμετρίας.

3.1 Στατικές και Δυναμικές Δομές Δεδομένων

Static δομές. Δομές που υποστηρίζουν ερωτήματα πάνω σε ένα αμετάβλητο σύνολο δεδομένων. Συνήθεις μετρικές είναι ο χρόνος κατασκευής T_{build} , ο χώρος S και ο χρόνος ερωτήματος T_{query} .

Dynamic δομές. Δομές που, επιπλέον των ερωτημάτων, υποστηρίζουν ενημερώσεις (updates). Ειδικότερα υποστηρίζουν εισαγωγή (insert) ή/και διαγραφή (delete) στοιχείων. Επιπρόσθετη μετρική είναι ο χρόνος ενημέρωσης T_{update} . Το κόστος των ενημερώσεων αναλύεται συνήθως ως ανάλυση χειρότερης περίπτωσης (worst-case), αναμενόμενης περίπτωσης (expected) ή επιμερισμένης ανάλυσης (amortized).

Σημειώσεις.

- Στην υπολογιστική γεωμετρία, οι δυναμικές δομές επιτρέπουν την τροποποίηση συνόλων σημείων ή τμημάτων και συχνά απαιτούν τη διατήρηση αμετάβλητων συνθηκών (invariants)· π.χ. σε δέντρα αναζήτησης απαιτείται “ισορροπία”.
- Ενδεικτικά αναφέρεται ότι υπάρχει και η κατηγορία των *streaming* δομών δεδομένων με τις οποίες δεν έχουμε ασχοληθεί.

3.2 Διαχρονικότητα (Persistence)

Η έννοια της διαχρονικότητας (persistence) κατηγοριοποιεί τον βαθμό διατήρησης παλαιότερων εκδόσεων μιας δομής και τον τρόπο με τον οποίο επιτρέπονται ερωτήματα ή ενημερώσεις σε αυτές. [DSS+89]

Κατηγορίες.

- *Non-persistent (Ephemeral)*: δεν διατηρούνται παλαιότερες εκδόσεις — κάθε ενημέρωση τροποποιεί την υπάρχουσα δομή και καταστρέφει την προηγούμενη μορφή.
- *Partial persistence*: επιτρέπονται ερωτήματα σε παλιές εκδόσεις, ενώ οι ενημερώσεις γίνονται μόνο στην πιο πρόσφατη έκδοση.
- *Full persistence*: επιτρέπονται και ερωτήματα και ενημερώσεις σε οποιαδήποτε έκδοση, με δυνατότητα διακλαδώσεων των εκδόσεων.
- *Confluent persistence*: περίπτωση της full persistence που επιπλέον επιτρέπει συγχώνευση (merge) δύο εκδόσεων για τη δημιουργία μίας νέας.

Σημειώσεις.

- persistent υλοποιήσεις συνήθως αυξάνουν το κόστος χώρου και/ή χρόνου των ενημερώσεων ανάλογα με τη μέθοδο υλοποίησης.

- Υπάρχουν πολλές παραλλαγές και βελτιστοποιήσεις μετατροπής εφήμερων (ephemeral) δομών σε persistent δομές. Κάποιες ενδεικτικά αποτελούν οι copy-on-write, fat nodes, path copying, nodes with boxes, snapshotting, persistent arrays, log structured d.s. και άλλες ή συνδυασμό μεθόδων (combination).
- Στην υπολογιστική γεωμετρία, persistent δομές χρησιμοποιούνται για την αποτελεσματική υλοποίηση αλγορίθμων που απαιτούν ιστορική πρόσβαση (π.χ. persistent segment trees για offline queries, plane-sweep με rollback κ.α.).

3.3 Succinct, compact and implicit δομές

Ο όρος εισήχθη από τον Jacobson το 1989. Οι δομές αυτές αποτελούν τρόπους αναπαράστασης δεδομένων που χρησιμοποιούν χώρο κοντά στο πληροφοριακό κατώφλι (information-theoretic lower bound), ενώ επιτρέπουν αποδοτικές λειτουργίες ερωτημάτων και ενημερώσεων. [Jac89]

Έστω Z ο ελάχιστος αριθμός bits που απαιτούνται θεωρητικά για την αναπαράσταση ενός συνόλου δεδομένων, τότε μια δομή δεδομένων μπορεί να χαρακτηριστεί ως:

- *Implicit*: αν χρησιμοποιεί $Z + O(1)$ bits.
- *Succinct*: αν χρησιμοποιεί $Z + o(Z)$ bits.
- *Compact*: αν χρησιμοποιεί $O(Z)$ bits.

4 Δενδρικές Δομές Δεδομένων

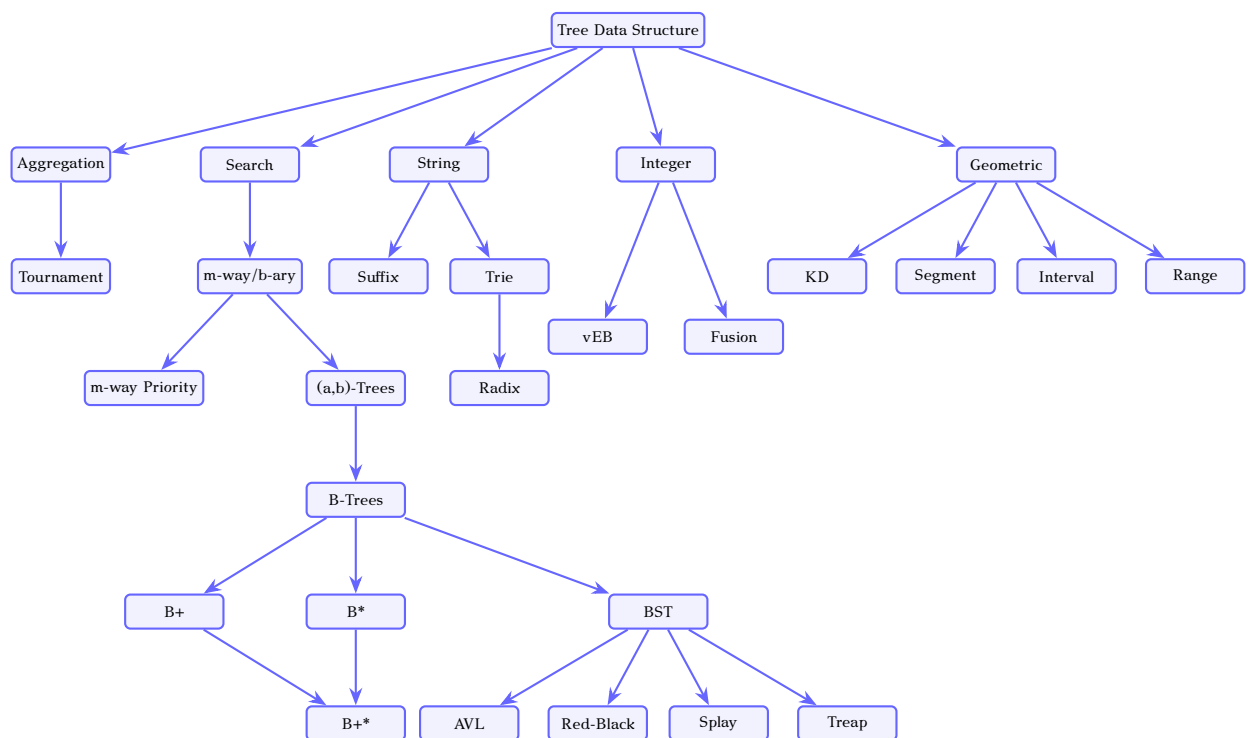
Οι δενδρικές δομές δεδομένων είναι θεμελιώδεις δομές που οργανώνουν δεδομένα ιεραρχικά, επιτρέποντας αποδοτικές πράξεις όπως αναζήτηση, εισαγωγή, διαγραφή και ενημέρωση. Η βασική ιδέα είναι η αναδρομική διάσπαση μιας συλλογής σε υποδέντρα, μειώνοντας την ανάγκη γραμμικών σαρώσεων και επιτυγχάνοντας συχνά λογαριθμικούς χρόνους για κρίσιμες λειτουργίες. Στόχος της ενότητας είναι να παρουσιάσει την ποικιλία των δεντρικών δομών με έμφαση στον σκοπό χρήσης και στα χαρακτηριστικά που τις κάνουν κατάλληλες για διαφορετικά προβλήματα και μοντέλα μνήμης.

Αρχικά θα εξετάσουμε τις δομές που προσανατολίζονται σε συσσώμωση (aggregation), με κύριο παράδειγμα τα tournament trees, τα οποία χρησιμοποιούνται για επαναλαμβανόμενες συγκρίσεις και για αποδοτική εύρεση ακροτάτων σε ροές δεδομένων. Το κύριο μέρος της ενότητας αφορά τα δέντρα αναζήτησης βασισμένα σε συγκρίσεις· εδώ θα παρουσιάσουμε την ιεραρχία από το γενικό προς το ειδικό, καλύπτοντας M-way / k-ary δέντρα και (a,b)-trees ως γενικεύσεις, καθώς και κλασικές υλοποιήσεις εξισορρόπησης όπως AVL και Red-Black, μαζί με παραλλαγές όπως treaps.

Θα παρουσιαστούν επίσης εξειδικευμένες κατηγορίες: δέντρα για συμβολοσειρές (string trees) όπως tries - που επιτρέπουν γρήγορη αναζήτηση προθεμάτων και υποσυμβολοσειρών, δομές για ακεραίους (integer trees) όπως van Emde Boas trees και fusion trees - που αξιοποιούν bit-level ιδιότητες.

4.1 Χάρτης Δενδρικών Δομών

Παρακάτω φαίνεται ο χάρτης με τις δενδρικές δομές δεδομένων ανά κατηγορία οι οποίες θα αναπτυχθούν στην παρούσα εργασία. Η κατεύθυνση της ακμής είναι από την γενικότερη προς την ειδικότερη δομή.



4.2 Aggregation trees

Τα *aggregation trees* είναι ριζωμένα δέντρα σχεδιασμένα να διατηρούν συνοπτικές πληροφορίες για υποδέντρα: κάθε εσωτερικός κόμβος αποθηκεύει μία τιμή που προκύπτει από τις τιμές των παιδιών μέσω μίας συνάρτησης συσσώρευσης (π.χ. $\max$, $\min$, $\sum$, count , ή γενικότερα κάποια associative/commutative συνάρτηση). Τα φύλλα περιέχουν τις ατομικές τιμές ή δείκτες σε εγγραφές· η τιμή της ρίζας αντιπροσωπεύει το aggregation ολόκληρου του δέντρου.

Βασικές λειτουργίες είναι η κατασκευή (*build*), η ενημέρωση τιμής σε φύλλο (*update*) και το ερώτημα συσσώρευσης για τη ρίζα ή οποιοδήποτε υποδέντρο (*aggregate query*). Στην τυπική περίπτωση ισορροπημένου δέντρου το μήκος ενός μονοπατιού από φύλλο προς και από την ρίζα κοστίζει $O(\log n)$ χρόνο. [CLR+09]

Σημείωση. Είναι σημαντικό ότι τα *aggregation trees* αποτελούν ειδική περίπτωση *unordered trees*: η σχετική αριστερά–δεξιά δέση των παιδιών δεν επηρεάζει το αποτέλεσμα του aggregation. Η ιδιότητα της μη διάταξης επιτρέπει ευελιξία στην υλοποίηση (π.χ. επανατοποθέτηση παιδιών χωρίς αλλαγή σημασίας), διευκολύνει *balanced layouts* για μικρότερο ύψος και συχνά απλοποιεί τεχνικές επαναδιανομής/συγχώνευσης μεταξύ γειτονικών κόμβων. Ως πρακτικό αποτέλεσμα, *aggregation / unordered trees* χρησιμοποιούνται ευρέως όπου χρειάζονται γρήγορα συνοπτικά δεδομένα.

4.2.1 Tournament trees

Ορισμός και περιγραφή. Τα *tournament trees* αποτελούν ειδική περίπτωση *aggregation trees*. Πιο συγκεκριμένα, ένα *tournament tree* καλείται ένα ριζωμένο δυαδικό δέντρο όπου τα φύλλα αντιστοιχούν σε στοιχεία και κάθε εσωτερικός κόμβος κρατά τον «νικητή» — το αποτέλεσμα μιας συνάρτησης των παιδιών του (π.χ. το μέγιστο ή το ελάχιστο). Έτσι, η ρίζα περιέχει τον συνολικό νικητή της διάταξης. [Knu98]

Winner trees και loser trees. Τα *tournament trees* διακρίνονται σε δύο κατηγορίες: *winner trees* (συνήθης επιλογή όταν αναφερόμαστε σε *tournament trees*) και *loser trees*. Συγκεκριμένα, στην περίπτωση των *winner trees* κάθε εσωτερικός κόμβος αποθηκεύει τον νικητή της σύγκρισης μεταξύ των δύο παιδιών του, οπότε η ρίζα περιέχει άμεσα τον συνολικό νικητή. Αντίθετα, στην περίπτωση ενός *loser tree*, οι εσωτερικοί κόμβοι αποθηκεύουν τον «χαμένο» κάθε σύγκρισης και ο τρέχων νικητής κρατιέται χωριστά (διευκρίνηση: στα *loser trees* πάλι συγκρίνονται οι νικητές, αλλά αποθηκεύεται ο ηττημένος). [Knu98]

Βασικές λειτουργίες.

- **Build(leaves):** κατασκευή του δέντρου από τα φύλλα (αρχικοί αγώνες).
- **Winner():** επιστροφή του στοιχείου στη ρίζα ($\max/\min$).
- **ExtractWinner():** αφαίρεση του νικητή και ενημέρωση του δέντρου (αντικατάσταση του αντίστοιχου φύλλου και επαναυπολογισμός κατά μήκος του μονοπατιού).
- **ReplaceLeaf(leaf, newValue)/Update:** ενημέρωση τιμής φύλλου και επαναυπολογισμός κόμβων στο μονοπάτι προς τη ρίζα.

Χρόνοι υλοποίησης. Έστω n ο αριθμός φύλλων και $h = \lceil \log_2 n \rceil$ το ύψος.

- Build: $\Theta(n)$ συγκρίσεις συνολικά.
- Winner(): $O(1)$.
- ExtractWinner/ReplaceLeaf: $O(h) = O(\log n)$ συγκρίσεις — απαιτείται επαναυπολογισμός κατά μήκος ενός μονοπατιού ρίζα προς φύλλο / φύλλο προς ρίζα.
- Χώρος: $O(n)$, συνήθως αποθηκεύεται σε πίνακα ως πλήρες δυαδικό δέντρο.

Σημείωση. Στο πλαίσιο της εξωτερικής ταξινόμησης (βλ. §7.5), το k -way merge μπορεί να υλοποιηθεί με (*loser*) tournament tree σε εξωτερική μνήμη, αποθηκεύοντας το δέντρο σε κόμβους/σελίδες μεγέθους $\Theta(B)$ και δουλεύοντας με μεταφορές block. [Knu98; Vit01] Επιπλέον, η ίδια ιδέα μπορεί να αποκτήσει buffered παραλλαγές, όπου ενημερώσεις/αντικαταστάσεις δεν προωθούνται άμεσα μέχρι χαμηλά επίπεδα, αλλά συσσωρεύονται σε buffers και εφαρμόζονται μαζικά, βελτιώνοντας το amortized I/O σε σχέση με την «άμεση» ενημέρωση ανά πράξη. [San99]

4.3 Search trees

Τα δέντρα αναζήτησης αποτελούν την κλασική απάντηση στο πρόβλημα του *ordered dictionary*: διατηρούν ένα δυναμικό σύνολο διατεταγμένων κλειδιών και υποστηρίζουν αποδοτικά search, insert, delete, καθώς και predecessor/successor και ερωτήματα εύρους (range queries). Στο *comparison model*, η επίδοση καθορίζεται κυρίως από το ύψος h του δέντρου: όσο μικρότερο είναι το h , τόσο ισχυρότερες είναι οι εγγυήσεις για το κόστος των πράξεων. Για τον λόγο αυτό, ο σχεδιασμός των search trees βασίζεται σε *invariants* ισορροπίας/πλήρωσης που διατηρούν το ύψος λογαριθμικό και επιτρέπουν «τοπικές» διορθώσεις μετά από ενημερώσεις (π.χ. rotations, split/merge/borrow). [CLR+09; MS08]

Στις επόμενες υποενότητες παρουσιάζουμε μια αντιπροσωπευτική κλιμάκωση ιδεών από το γενικό προς το ειδικό. Ξεκινάμε από τα *multi-way* (m -ary) δέντρα, όπου κάθε κόμβος κρατά πολλαπλά κλειδιά και έχει μεγάλο βαθμό διακλάδωσης· αυτή η επιλογή μειώνει το ύψος και είναι ιδιαίτερα φυσική όταν το κόστος κυριαρχείται από προσπελάσεις σε blocks (εξωτερική μνήμη). [MS08; BM72] Έπειτα περνάμε στα (a, b) -trees, μια γενική οικογένεια ισορροπημένων multi-way δέντρων με αυστηρά όρια βαθμού, που λειτουργεί ως θεωρητικό υπόβαθρο για τα B-trees και τις παραλλαγές τους. [MS08; BM72]

Στη συνέχεια εστιάζουμε σε *balanced binary search trees*: τα AVL επιβάλλουν αυστηρότερη ισορροπία ύψους, ενώ τα Red-Black χρησιμοποιούν πιο «χαλαρό» invariant (με χρωματισμό) ώστε οι ενημερώσεις να είναι απλούστερες, και στις δύο περιπτώσεις με εγγυημένο $O(\log n)$ worst-case κόστος για βασικές πράξεις. [AL62; GS78; CLR+09] Τέλος, παρουσιάζουμε τα *treaps*, τα οποία συνδυάζουν ιδιότητες BST και heap μέσω τυχαίων priorities, δίνοντας φυσικές πράξεις split/merge και αναμενόμενο $O(\log n)$ ύψος. [SA96]

4.3.1 Multi-way Trees

Ορισμός και περιγραφή. Multi-way trees (m -way ή m -ary trees) ονομάζεται κάθε δέντρο αναζήτησης στο οποίο κάθε κόμβος μπορεί να έχει μέχρι και m παιδιά (branching factor m). Κάθε κόμβος αποθηκεύει έως $m - 1$ κλειδιά σε αύξουσα σειρά που χωρίζουν τα υποδέντρα του. [MS08]

Βασικές λειτουργίες.

- `search(key)`: αναζήτηση κλειδιού.
- `insert(key)`: εισαγωγή νέου κλειδιού.
- `delete(key)`: διαγραφή υπάρχοντος κλειδιού.
- `predecessor / successor`: εύρεση προηγούμενου ή επόμενου κλειδιού.
- `range_query`: ερωτήματα εύρους μεταξύ δύο ορίων.
- `traversal`: διάσχιση του δέντρου σε ταξινομημένη σειρά.

Χρονική / χωρική πολυπλοκότητα. Έστω n το πλήθος των στοιχείων και m ο μέγιστος αριθμός παιδιών ανά κόμβο.

- Ύψος (σε ισορροπημένη μορφή): $h = \Theta(\log_m n)$. [MS08]
- Αναζήτηση / εισαγωγή / διαγραφή: $O(h)$ επισκέψεις κόμβων.
- Κόστος συγκρίσεων ανά κόμβο: γραμμική σάρωση $O(m)$ ή binary search $O(\log m)$ στο RAM μοντέλο (αν γίνεται binary search εντός κόμβου).
- External-memory (B-tree): για $m = \Theta(B)$ (όπου B το μέγεθος block) το κόστος σε I/O για `search/insert/delete` είναι $\Theta(\log_B n)$. [BM72; AV88; Vit01]
- Χώρος: $O(n)$ για την αποθήκευση όλων των στοιχείων και δεικτών.

Σημείωση. Κάποιες φορές στη βιβλιογραφία τα k -ary δέντρα αναφέρονται ως m -ary δέντρα. Επίσης, σε ορισμένες περιπτώσεις ο όρος χρησιμοποιείται για δέντρα όπου κάθε εσωτερικός κόμβος έχει ακριβώς m παιδιά.

4.3.2 (a,b)-trees

Ορισμός και περιγραφή. Ένα (a, b) -tree είναι ένα ριζωμένο, ισορροπημένο δέντρο αναζήτησης με παραμέτρους a, b που ικανοποιούν $2 \leq a \leq \frac{b+1}{2}$. Κύρια invariants: κάθε εσωτερικός κόμβος (εκτός της ρίζας) έχει τουλάχιστον a και το πολύ b παιδιά, η ρίζα έχει ≥ 2 και $\leq b$ παιδιά (εκτός της περίπτωσης μικρού δέντρου), όλα τα φύλλα έχουν το ίδιο βάθος, και τα κλειδιά σε κάθε κόμβο είναι ταξινομημένα και χωρίζουν αντίστοιχα τα υποδέντρα. [MS08]

Σχέση με B-trees. Ειδική περίπτωση των (a, b) -trees αποτελούν τα **B-trees**, όπου ένα B-tree τάξης b είναι ένα (a, b) -tree με $a = \lceil b/2 \rceil$. [BM72; MS08]

Βασικές λειτουργίες.

- `search(key)`: αναζήτηση κλειδιού.
- `insert(key)`: εισαγωγή νέου κλειδιού — σε υπερχείλιση κόμβου γίνεται split και, αν χρειαστεί, επανακατανομή ή προώθηση μέχρι τη ρίζα.
- `delete(key)`: διαγραφή κλειδιού — μετά τη διαγραφή διορθώνεται το πλήθος παιδιών/κλειδιών με borrow (δανεισμό από γείτονα) ή merge (συγχώνευση).
- `predecessor/successor`: εύρεση προηγούμενου / επόμενου κλειδιού.
- `range_query`, `traversal`, καθώς και λειτουργίες `join/split` σε επίπεδο υποδέντρων.

Χρονική / χωρική πολυπλοκότητα. Έστω n ο αριθμός κλειδιών.

- Ύψος: $h = \Theta(\log n)$. Ειδικότερα, έστω L το πλήθος των φύλλων, ισχύει $b^h \geq L$ και $a^{h-1} \leq L$, άρα $\log_b L \leq h \leq 1 + \log_a L$. [MS08]
- Αναζήτηση / εισαγωγή / διαγραφή (σε όρους προσβάσεων κόμβων): $O(h)$ node accesses.
- Κόστος συγκρίσεων ανά κόμβο: γραμμική σάρωση $O(b)$ ή binary search $O(\log b)$ εντός κόμβου.
- External-memory: για $b = \Theta(B)$ (όπου B το μέγεθος block) το κόστος σε I/O για search/insert/delete είναι $\Theta(\log_B n)$. [BM72; AV88; Vit01]
- Χώρος: $\Theta(n)$ συνολικά — n κλειδιά και $\Theta(n)$ δείκτες.

Σημείωση. Το βασικό πλεονέκτημα των (a, b) -trees είναι ότι εξασφαλίζουν ισχυρά invariants πλήρωσης που περιορίζουν το ύψος και απλοποιούν τις διαδικασίες split/merge/borrow. [MS08]

4.3.3 Binary Search Trees: AVL και Red-Black trees

AVL trees. Τα AVL trees είναι *height-balanced* BSTs: για κάθε κόμβο, οι ύψη των δύο υποδέντρων του διαφέρουν το πολύ κατά 1. Ως αποτέλεσμα, το ύψος είναι $O(\log n)$ και οι πράξεις search/insert/delete εκτελούνται σε $O(\log n)$ στη χειρότερη περίπτωση (με rotations για αποκατάσταση ισορροπίας). [AL62]

Red-Black trees. Τα Red-Black trees αποτελούν balanced BSTs που κωδικοποιούν πληροφορία ισορροπίας με 1 bit ανά κόμβο (χρώμα) και επιτυγχάνουν ύψος $O(\log n)$, άρα search/insert/delete σε $O(\log n)$ worst-case. [GS78; CLR+09]

4.3.4 Treaps

Ορισμός και περιγραφή. Τα *treaps* είναι τυχαιοποιημένα BSTs που συνδυάζουν: (i) ιδιότητα *BST* ως προς το *key* και (ii) ιδιότητα *heap* ως προς μία *priority*. Οι priorities θεωρούνται ανεξάρτητα τυχαία δείγματα (i.i.d.), και υπό αυτή την υπόθεση το σχήμα του treap έχει την ίδια κατανομή με τυχαίο BST από τυχαία σειρά εισαγωγής. [SA96]

Βασικές λειτουργίες.

- `search(key)`: αναζήτηση ενός κλειδιού.
- `insert(key)`: εισαγωγή νέου κλειδιού (με τυχαία priority).
- `delete(key)`: διαγραφή υπάρχοντος κλειδιού.
- `split(T, key)`: διάσπαση του T σε δύο δέντρα με βάση το *key*.
- `merge(T1, T2)`: ένωση δύο treaps $T1, T2$ (όλα τα κλειδιά του $T1$ μικρότερα από αυτά του $T2$).
- `find_kth(k)` / `order_statistics`: εύρεση του k -οστού στοιχείου.

Χρονική / χωρική πολυπλοκότητα. Για n κόμβους, υπό την υπόθεση ανεξάρτητων τυχαίων priorities, το ύψος είναι αναμενόμενο $O(\log n)$ και οι `search/insert/delete/split/merge` εκτελούνται σε αναμενόμενο $O(\log n)$. Ο χώρος είναι $O(n)$. [SA96].

Σημείωση. Ένα *treap* μπορεί να ιδωθεί ως ειδική περίπτωση *Cartesian tree*: αν ταξινομήσουμε τα κλειδιά $x_1 < \dots < x_n$ και θεωρήσουμε τις προτεραιότητες p_i , τότε το (μοναδικό, όταν όλα τα p_i είναι διακριτά) δυαδικό δέντρο που έχει *inorder* διάσχιση $x_1, \dots, x_n$ και ικανοποιεί την ιδιότητα *heap* ως προς τις προτεραιότητες είναι ακριβώς το *Cartesian tree* της ακολουθίας $(p_1, \dots, p_n)$. Με άλλα λόγια, τα *treaps* είναι *Cartesian trees* όπου οι προτεραιότητες επιλέγονται (συνήθως) τυχαία.

4.4 String trees

Τα *string trees* είναι δενδρικές δομές δεδομένων για αποδοτική αναπαράσταση και επεξεργασία συμβολοσειρών. Κεντρική ιδέα τους είναι η αξιοποίηση κοινών προθεμάτων (ή, σε άλλες παραλλαγές, κοινών υποσυμβολοσειρών), ώστε να αποφεύγεται η επανάληψη πληροφορίας και να υποστηρίζονται γρήγορα ερωτήματα όπως *search* και *prefix queries*. Ανάλογα με τη δομή, προκύπτουν διαφορετικά trade-offs μεταξύ χρόνου, χώρου και πολυπλοκότητας υλοποίησης. [Gus97]

Στην παρούσα υποενότητα εστιάζουμε αποκλειστικά στα *tries* (prefix trees), τα οποία αποθηκεύουν ένα σύνολο λέξεων πάνω σε αλφάβητο Σ ως εξής: κάθε ακμή φέρει ένα σύμβολο, και κάθε κόμβος αντιστοιχεί στο πρόδεμα που σχηματίζεται από τις ετικέτες στη διαδρομή από τη ρίζα. Με αυτόν τον τρόπο, η αναζήτηση μιας λέξης (και γενικότερα η αναζήτηση προθέματος) ανάγεται σε διάσχιση μήκους ίσου με το μήκος της συμβολοσειράς, ενώ η κοινή προθεματική δομή μοιράζεται μεταξύ διαφορετικών λέξεων. [Gus97]

4.4.1 Tries

Ένα *Trie* (prefix tree) για ένα σύνολο συμβολοσειρών S αποτελεί ένα ριζωμένο δέντρο όπου κάθε ακμή φέρει έναν χαρακτήρα και κάθε κόμβος αντιστοιχεί στο πρόδεμα που προκύπτει από το μονοπάτι από τη ρίζα έως τον κόμβο. Η ρίζα αναπαριστά το κενό πρόδεμα και ορισμένοι κόμβοι σημειώνονται ως τερματικοί όταν το πρόδεμα είναι πλήρης λέξη του S . [Fre60; Knu98; Gus97]

Βασικές λειτουργίες

- `insert(s)`: εισαγωγή της συμβολοσειράς s : διασχίζουμε/δημιουργούμε κόμβους για κάθε χαρακτήρα και σημειώνουμε τον τελικό κόμβο.
- `search(s)`: έλεγχος αν $s \in S$: διασχίζουμε το *trie* χαρακτήρα–χαρακτήρα· επιτυχία αν ο τελικός κόμβος είναι τερματικός.
- `prefix_query(p)`: εύρεση/μέτρηση λέξεων με πρόδεμα p : διασχίζουμε μέχρι το p και κάνουμε *traversal* του υποδέντρου για εξαγωγή ή μέτρηση.
- `delete(s)`: διαγραφή της s : αποσημαίνουμε τον τελικό κόμβο και προαιρετικά καταργούμε περιττούς κόμβους.

Χρονική Πολυπλοκότητα

Για συμβολοσειρά μήκους $|s|$ (και αλφάβητο μεγέθους $|\Sigma|$):

- `insert(s)`: $O(|s|)$.

- $\text{search}(s): O(|s|)$.
- $\text{prefix_query}(p): O(|p| + k)$, όπου k το μέγεθος της εξόδου.
- $\text{delete}(s): O(|s|)$ (συν κόστος pruning αν γίνεται).

Οι χρόνοι είναι σε χαράκτες· το κόστος ανά βήμα εξαρτάται από το πώς αναπαριστώνται τα παιδιά. [Gus97]

Χωρική Πολυπλοκότητα

Χειρότερη περίπτωση: $O(\sum_{w \in S} |w|)$ κόμβοι/ακμές (πιο οικονομικά στην πράξη λόγω κοινών προδεμάτων). [Gus97]

Υλοποιητικά ζητήματα & trade-offs

- *Αναπαράσταση παιδιών*: πίνακας ($O(1)$ πρόσβαση, υψηλή μνήμη); hash map (συμπαγής, αναμεν. $O(1)$); balanced map ($O(\log |\Sigma|)$ εγγυημένο).
- *Compressed trie / Radix / Patricia*: συγχωνεύουν αλυσίδες μοναδικού παιδιού σε ετικέτες πολλαπλών χαρακτήρων, μειώνοντας κόμβους και βελτιώνοντας constants ενώ διατηρούν τους χρόνους $O(|s|)$ στη λειτουργία. [Mor68]
- *Πρόσθετα πεδία*: μετρητές υποδεμένων ή τιμές στα τερματικά διευκολύνουν `count_prefix` και άλλες ερωτήσεις σε $O(|p|)$.

4.5 Integer Trees

Τα *δέντρα ακεραίων* (*integer trees*) είναι δομές δεδομένων για κλειδιά που ανήκουν σε ένα πεπερασμένο σύμπαν ακεραίων τιμών, π.χ. $U = \{0, \dots, U-1\}$. Σε αντίθεση με τις καθαρά comparison-based δομές, εκμεταλλεύονται το μοντέλο *word-RAM* και τις bitwise/word-level πράξεις (shifts, masks κ.λπ.) ώστε να επιτυγχάνουν χρόνους που υπερβαίνουν τα κλασικά όρια των συγκρίσεων. [Hag98; FW93] Οι τεχνικές αυτές είναι ιδιαίτερα αποτελεσματικές σε λειτουργίες *ordered dictionary*, με έμφαση σε predecessor/successor, και σε παραλλαγές rank/select όταν η αναπαράσταση σε bits προσφέρει πρόσθετη αλγοριθμική πληροφορία. [Emd77; FW93]

Στις επόμενες υποενότητες παρουσιάζονται δύο κεντρικά παραδείγματα. Το *van Emde Boas (vEB) tree* βασίζεται σε αναδρομική αποσύνθεση του σύμπαντος και υποστηρίζει βασικές πράξεις σε $O(\log \log U)$ χρόνο. [Emd77] Τα *fusion trees*, από την άλλη, αξιοποιούν word-level bit manipulation και «παράλληλες» συγκρίσεις ώστε να μειώσουν το κόστος αναζήτησης σε $O(\frac{\log n}{\log w})$, όπου w είναι το μήκος λέξης. [FW93; FW94]

4.5.1 van Emde Boas Tree

Ορισμός και περιγραφή.

Ένα *van Emde Boas tree (vEB)* είναι μια αναδρομική δενδρική δομή δεδομένων που αποθηκεύει ένα δυναμικό σύνολο ακεραίων κλειδιών από ένα πεπερασμένο σύμπαν $U = \{0, 1, \dots, u-1\}$ (όπου $|U| = u$). Σχεδιάστηκε για την υποστήριξη πράξεων *ordered dictionary*, και ιδίως των successor και predecessor, σε χρόνο $O(\log \log u)$. [Emd77; CLR+09]

Η βασική ιδέα είναι η αναδρομική αποσύνθεση του σύμπαντος U σε $\ell = \lceil \sqrt{u} \rceil$ συστάδες. Κάθε κλειδί x γράφεται ως ζεύγος $(\text{high}(x), \text{low}(x))$, όπου $\text{high}(x)$ δείχνει σε ποια συστάδα ανήκει και $\text{low}(x)$ τη θέση του μέσα σε αυτή. Ο κόμβος διατηρεί (α) min και max

για άμεσους ελέγχους/απαντήσεις, (β) μια δομή *summary* που αποθηκεύει ποιες συστάδες είναι μη-κενές, και (γ) για κάθε συστάδα μια αναδρομική vEB δομή για τα αντίστοιχα low-κλειδιά. Έτσι, για πράξεις όπως *successor/predecessor* πρώτα ελέγχουμε αν υπάρχει απάντηση μέσα στην ίδια συστάδα και, αν όχι, βρίσκουμε μέσω του *summary* την επόμενη μη-κενή συστάδα και επιστρέφουμε το min της. Επειδή σε κάθε επίπεδο το μέγεθος του σύμπαντος μειώνεται από u σε $\sqrt{u}$, το βάθος της αναδρομής είναι $O(\log \log u)$. [Emd77; CLR+09]

Λειτουργίες.

- **Member/Find(x):** έλεγχος αν $x \in S$.
- **Insert(x):** εισαγωγή του x στο S .
- **Delete(x):** διαγραφή του x από το S .
- **Successor(x):** επιστρέφει το ελάχιστο $y \in S$ με $y > x$ (αν υπάρχει).
- **Predecessor(x):** επιστρέφει το μέγιστο $y \in S$ με $y < x$ (αν υπάρχει).
- **Min()/Max():** επιστρέφει το ελάχιστο/μέγιστο στοιχείο του S (αν $S \neq \emptyset$).

Χρονική Πολυπλοκότητα.

Για σύμπαν μεγέθους u , οι Find/Insert/Delete/Successor/Predecessor υλοποιούνται σε $O(\log \log u)$ (χειρότερη περίπτωση), ενώ οι Min/Max σε $O(1)$. Η κλασική υλοποίηση απαιτεί χώρο $\Theta(u)$. [Emd77; CLR+09]

Αποσύνθεση στοιχείων του σύμπαντος.

Έστω $U = \{0, 1, \dots, u-1\}$ και $\ell = \lceil \sqrt{u} \rceil$. Ορίζουμε [CLR+09]

$$\text{high}(x) = \left\lfloor \frac{x}{\ell} \right\rfloor, \quad \text{low}(x) = x \bmod \ell, \quad \text{index}(i, j) = i \cdot \ell + j.$$

Αλγόριθμος Successor(x):

Algorithm 1: Successor(V, x)

```

1  $i = \text{high}(x)$ ;
2 if  $\text{low}(x) < V.\text{cluster}[i].\text{max}$  then
3    $j = \text{Successor}(V.\text{cluster}[i], \text{low}(x))$ ;
4 else
5    $i = \text{Successor}(V.\text{summary}, \text{high}(x))$ ;
6  $j = V.\text{cluster}[i].\text{min}$ ;
7 return  $\text{index}(i, j)$ ;
```

Αλγόριθμος $\text{Insert}(x)$:

Algorithm 2: $\text{Insert}(V, x)$ (do not store min recursively)

```
1 if  $V.\text{min} == \text{None}$  then
2    $V.\text{min} = V.\text{max} = x$ ; //  $O(1)$  time
3   return;
4 if  $x < V.\text{min}$  then
5   swap( $x \leftrightarrow V.\text{min}$ );
6 if  $x > V.\text{max}$  then
7    $V.\text{max} = x$ ;
8 if  $V.\text{cluster}[\text{high}(x)].\text{min} == \text{None}$  then
9    $\text{Insert}(V.\text{summary}, \text{high}(x))$ ; // First Call
10  $\text{Insert}(V.\text{cluster}[\text{high}(x)], \text{low}(x))$ ; // Second Call
```

Χρονική Πολυπλοκότητα: $O(\log \log u)$.

Σε κάθε επίπεδο γίνονται $O(1)$ διεργασίες και το πολύ μία αναδρομική κλήση σε σύμπαν $\sqrt{u}$, άρα $T(u) = T(\sqrt{u}) + O(1) \Rightarrow T(u) = O(\log \log u)$. [Emd77; CLR+09]

4.6 Fusion Trees

Ορισμός και περιγραφή. Τα *Fusion Trees*, που εισήχθησαν από τους Fredman και Willard (1990), αποτελούν μια επαναστατική δομή δεδομένων καθώς κατάφεραν να σπάσουν το φράγμα του $\Omega(\log n)$ το οποίο ισχύει για συγκρίσεις, στο comparison model [FW93]. Η δομή αντιστοιχεί στο μοντέλο *Word-RAM* με μέγεθος λέξης w bits και εκμεταλλεύονται τον παραλληλισμό σε επίπεδο bit (word-level parallelism) για να επιτύχουν χρόνο αναζήτησης:

$$O(\log_w n) = O\left(\frac{\log n}{\log w}\right).$$

Υιοθετώντας την καθιερωμένη *transdichotomous* παραδοχή ότι το μέγεθος της λέξης είναι τουλάχιστον λογαριθμικό ως προς το μέγεθος της εισόδου ($w \geq \log n$), η πολυπλοκότητα γίνεται:

$$O\left(\frac{\log n}{\log \log n}\right),$$

γεγονός που τα καθιστά ασυμπτωτικά ταχύτερα από τα κλασικά δυαδικά δέντρα αναζήτησης.

Δομή και Βαθμός Διακλάδωσης. Ένα Fusion Tree είναι ουσιαστικά ένα *B-tree* με προσεκτικά επιλεγμένο βαθμό διακλάδωσης (branching factor) $B = \Theta(w^{1/5})$. Το ύψος του δέντρου προκύπτει άμεσα από τις ιδιότητες των B-trees:

$$h = \log_B n = \frac{\log n}{\log(w^{1/5})} = 5 \cdot \frac{\log n}{\log w} = \Theta\left(\frac{\log n}{\log w}\right).$$

Το κλειδί της επιτυχίας είναι ότι, παρόλο που το B μπορεί να είναι μεγάλο, η αναζήτηση μέσα σε κάθε κόμβο (node search) εκτελείται σε σταθερό χρόνο $O(1)$, αντί για $O(\log B)$.

Η Τεχνική του Sketching. Για να πετύχουμε αναζήτηση σε $O(1)$ μέσα στον κόμβο, πρέπει να συγκρίνουμε το κλειδί αναζήτησης q με όλα τα k κλειδιά του κόμβου ταυτόχρονα. Επειδή όμως k κλειδιά των w bits απαιτούν $k \cdot w$ bits (που δεν χωράνε σε μια λέξη), χρησιμοποιούμε τη συμπίεση *Sketching*.

Η ιδέα βασίζεται στη γεωμετρία των κλειδιών αν τα φανταστούμε σε ένα δυαδικό δέντρο (trie):

- *Splitting Nodes*: Αν έχουμε k κλειδιά, τα μονοπάτια τους στο trie διαχωρίζονται σε το πολύ $k-1$ κόμβους διακλάδωσης. Αυτά είναι τα μόνα σημεία (bits) που έχουν σημασία για τη διάταξη.
- *Distinguishing Bits*: Έστω $b_0, b_1, \dots, b_{r-1}$ οι θέσεις των bits που αντιστοιχούν σε αυτούς τους κόμβους διακλάδωσης ($r < k$).
- *Sketch*: Για κάθε κλειδί x , κρατάμε μόνο τα bits σε αυτές τις θέσεις. Ουσιαστικά, κάθε bit του sketch κωδικοποιεί αν το κλειδί πήγε *αριστερά* (0) ή *δεξιά* (1) στον αντίστοιχο κόμβο διακλάδωσης.

Το συνολικό μέγεθος όλων των sketches στον κόμβο είναι περίπου $k \cdot r \approx k^2 = (w^{1/5})^2 = w^{2/5}$ bits. Εφόσον $w^{2/5} \ll w$, όλα τα συμπιεσμένα κλειδιά χωράνε άνετα σε μία λέξη μνήμης, επιτρέποντας την παράλληλη σύγκριση.

Αναζήτηση στον Κόμβο (Parallel Comparison). Η αναζήτηση γίνεται με χρήση ειδικών εντολών πολλαπλασιασμού και λογικών πράξεων (bitwise AND, OR) που επιτρέπουν την ταυτόχρονη σύγκριση του sketch του q με όλα τα sketches του κόμβου. Επειδή το sketching χάνει πληροφορία, η διαδικασία επιστρέφει ένα "κατά προσέγγιση" αποτέλεσμα, το οποίο διορθώνεται στο τέλος (despying).

Algorithm 3: Fusion Tree Node Search (Simplified)

Input: Node v containing keys $x_1 \dots x_k$, Query key q
Result: The child pointer corresponding to the range of q

```

1  $s_q \leftarrow \text{Sketch}(q)$ ; // Extract only the relevant bits of  $q$ 
2  $S_{node} \leftarrow v.\text{all\_sketches}$ ; // Packed sketches of all keys
   // Parallel comparison in  $O(1)$  using fusion arithmetic
3  $\text{match\_index} \leftarrow \text{ParallelCompare}(s_q, S_{node})$ ;
   // Identify candidate child
4  $i \leftarrow \text{MostSignificantBit}(\text{match\_index})$ ;
5  $\text{candidate} \leftarrow v.\text{children}[i]$ ;
   // Refinement step (Despying)
6 if  $q$  falls within range of candidate then
7   | return candidate;
8 else
9   |  $\text{correction} \leftarrow \text{FindRange}(v, q, i)$ ; // Check neighbors ( $O(1)$ )
10  | return correction;
11 end
```

5 Computational-Geometry Trees

Οι δενδρικές δομές δεδομένων της υπολογιστικής γεωμετρίας αποτελούν ειδική κατηγορία δομών δεδομένων σχεδιασμένων για την οργάνωση και επεξεργασία γεωμετρικών αντικειμένων (σημεία, τμήματα, ορθογώνια, κλπ.). Στόχος τους είναι η αποτελεσματική εξυπηρέτηση γεωμετρικών ερωτημάτων—όπως *orthogonal range queries*, *nearest-neighbor*, *stabbing/intersection queries* και *reporting/counting*—υπό περιορισμούς χώρου και χρόνου. Τα δέντρα αυτά συνδυάζουν ιδέες από δέντρα αναζήτησης, διαμερισμό χώρου και αρχιοδότηση σε στρώματα (*layering*), και συχνά ενσωματώνουν τεχνικές όπως *fractional cascading*, *persistence* και *rank-space reduction* για να βελτιώσουν χρόνους και χρήση μνήμης. Η σχεδιάσή τους διαφοροποιείται ανάλογα με το αν το πρόβλημα είναι στατικό ή δυναμικό, αν το σύνολο δεδομένων χωράει στην κύρια μνήμη ή απαιτεί παραλλαγές εξωτερικής μνήμης και σε σχέση με την διάσταση $d \in \mathbb{N}$ του χώρου. [Sha78; BCK+08; CG86; DSS+89; BF79]

Στις υπόλοιπες υποενότητες θα παρουσιαστούν τα βασικά, χαρακτηριστικά δέντρα και οι τεχνικές χρήσης τους. Το *Interval tree* παρέχει αποδοτικές λειτουργίες για την εύρεση τμημάτων που τέμνουν ένα σημείο ή ένα άλλο τμήμα. Το *Segment tree* είναι δομή για την αποθήκευση τμημάτων/διαστημάτων και εξυπηρετεί εύκολα *stabbing queries* και *range-reporting* κατά μήκος μίας διάστασης. Το *Kd-tree* οργανώνει σημεία του $\mathbb{R}^d$ για *nearest-neighbor* και *orthogonal range queries*, με απλές και πρακτικές υλοποιήσεις για μικρό d . Το *Range tree* είναι ένα *layered* δέντρο για ορθογώνιες (*orthogonal*) αναζητήσεις σε πολλές διαστάσεις, προσφέροντας θεμελιώδεις εγγυήσεις τύπου $O(\log^d n + k)$ για *reporting* (με αντίστοιχα trade-offs στο χώρο). [BCK+08; BF79] Κάθε μία από αυτές τις δομές θα αναλυθεί ως προς τις αμετάβλητες συνθήκες (*invariants*), τις βασικές λειτουργίες και την ασυμπτωματική πολυπλοκότητά τους.

5.1 Interval tree

Ορισμός και υλοποίηση. Το *interval tree* είναι δομή για την αποθήκευση ενός συνόλου διαστημάτων $[l, r]$ σε μία διάσταση (ή περισσότερες υπο κάποιες παραδοχές), με στόχο την αποδοτική απάντηση ερωτημάτων επικάλυψης και τομής (*stabbing*). [BCK+08; BF79]

Η *κεντροποιημένη* (*centered*) υλοποίηση ενός *interval tree* είναι η εξής: έστω S το σύνολο διαστημάτων, επιλέγουμε ένα *κεντρικό* σημείο c_v (συνήθως η διάμεσος όλων των άκρων των διαστημάτων του συνόλου). Στον κόμβο v αποθηκεύονται αποκλειστικά τα διαστήματα που περιέχουν το c_v . Τα υπόλοιπα διαστήματα που βρίσκονται εξολοκλήρου αριστερά ή δεξιά του c_v , προωθούνται αναδρομικά στο αριστερό υποδέντρο και δεξί υποδέντρο αντίστοιχα. Για αποδοτικό *reporting*, τα διαστήματα που αποθηκεύονται στον κόμβο διατηρούνται σε δύο βοηθητικές λίστες: η μία είναι ταξινομημένη κατά το αριστερό άκρο σε αύξουσα σειρά και η άλλη κατά το δεξί άκρο σε φθίνουσα σειρά· αυτή η διπλή ταξινόμηση επιτρέπει γρήγορο τερματισμό της σάρωσης όταν δεν ικανοποιούνται πλέον οι συνθήκες επικάλυψης. Η κατασκευή γίνεται επαναληπτικά: επιλέγουμε c_v , διανέμουμε τα διαστήματα σε «περιέχουν c_v / αριστερά / δεξιά» και κτίζουμε αναδρομικά τα υποδέντρα. [BCK+08]

Βασικές λειτουργίες.

- `build(S)`: κατασκευή της *centered interval tree* από το σύνολο S .
- `stabbing_query(q)`: επιστροφή όλων των διαστημάτων που περιέχουν το σημείο q .
- `overlap_query([a, b])`: επιστροφή όλων των διαστημάτων που επικαλύπτονται με το διάστημα $[a, b]$.

- (Update) insert/delete: σε στατικές εφαρμογές αποφεύγονται· σε δυναμικές χρήσεις απαιτούν αναδόμηση/ενημέρωση των λιστών στον κόμβο και πιθανή επανακατασκευή υποδέντρου.

Χρονική και χωρική πολυπλοκότητα.

- $\text{build}(S): O(n \log n)$ τυπικά (ταξινόμηση άκρων + επαναλαμβανόμενη διανομή). [BCK+08]
- $\text{stabbing_query}(q): O(\log n + k)$, όπου k ο αριθμός των αναφερόμενων διαστημάτων. [BCK+08]
- $\text{overlap_query}([a, b]): O(\log n + k)$ με ανάλογη λογική (χρήση και των δύο λιστών στους σχετικούς κόμβους για early-exit και αναδρομική διεύθυνση προς υποδέντρα). [BCK+08]
- Χώρος: $O(n)$ — κάθε διάστημα αποθηκεύεται σε ακριβώς έναν κόμβο (ο κόμβος του κέντρου που το περιέχει). [BCK+08]
- insert/delete: σε δυναμική περίπτωση είναι ακριβότερες και συνήθως απαιτούν τοπική τροποποίηση της λίστας στο κατάλληλο κόμβο ή ενδεχόμενη επανακατασκευή υποδέντρου· το κόστος μπορεί να φτάσει $O(m)$ όπου m το μέγεθος του υποδέντρου που αναδομείται, γι' αυτό σε δυναμικά περιβάλλοντα προτιμάται περιοδικό rebuild (lazy rebuilding).

Σημειώσεις.

- Η επιλογή του κεντρικού σημείου c_v (π.χ. διάμεσος των άκρων) είναι κρίσιμη για ισορροπία: κατάλληλη επιλογή εξασφαλίζει $O(\log n)$ βάθος με τυπικές κατανομές. [BCK+08]
- Η centered υλοποίηση είναι ιδιαίτερα πρακτική για στατικά σύνολα όπου πολλά queries απαιτούν γρήγορο reporting· για πολύ δυναμικά σενάρια απαιτείται μηχανισμός περιοδικής αναδόμησης ή άλλη πολιτική ενημέρωσης.

5.2 Segment tree

Ορισμός και Κατασκευή. Ένα *segment tree* χρησιμοποιείται για την αποδοτική διαχείριση και αναζήτηση διαστημάτων στην μία διάσταση (με γενικεύσεις και σε περισσότερες διαστάσεις). Η βασική δομή πάνω στην οποία κατασκευάζεται είναι ένα *balanced binary tree*. Έστω I ένα σύνολο διαστημάτων της μορφής $[l, r]$. [BCK+08; BF79]

Από το σύνολο I κατασκευάζουμε το σύνολο όλων των διαφορετικών άκρων και αφού τα ταξινομήσουμε, ορίζουμε τα *elementary intervals* ως εξής: $(-\infty, p_1), [p_1, p_1], (p_1, p_2), [p_2, p_2], \dots, [p_m, p_m], (p_m, +\infty)$. Τα *elementary intervals* αποθηκεύονται στα φύλλα ενός *balanced binary tree* με τη φυσική τους σειρά. Σε κάθε εσωτερικό κόμβο v του δέντρου αναθέτουμε ένα διάστημα $\text{Int}(v)$, το οποίο ορίζεται ως η ένωση των διαστημάτων που αντιστοιχούν στα φύλλα του υποδέντρου με ρίζα τον κόμβο v . [BCK+08]

Αφού κατασκευαστεί το δέντρο, κάθε αρχικό διάστημα $s \in I$ εισάγεται ως εξής: ξεκινώντας από τη ρίζα, αν το $\text{Int}(v)$ ενός κόμβου v περιέχεται πλήρως στο s , τότε το s αποθηκεύεται στον κόμβο v . Διαφορετικά, η διαδικασία συνεχίζεται αναδρομικά στα παιδιά του v των οποίων τα αντίστοιχα διαστήματα τέμνουν το s . Με αυτόν τον τρόπο κάθε διάστημα αποθηκεύεται στους κόμβους του βρίσκονται όσο πιο κοντά στην ρίζα γίνεται και το s καλύπτει πλήρως το $\text{Int}(v)$ αλλά όχι το $\text{Int}(\text{parent}(v))$. [BCK+08]

Υποστηριζόμενες λειτουργίες.

- `build(S)`: δημιουργία των endpoints, elementary intervals και του δέντρου.
- `insert([l,r])`: εισαγωγή διαστήματος (αποθήκευση αναφοράς στις λίστες των canonical κόμβων).
- `delete([l,r])`: αφαίρεση (απαιτεί handle/ID ή lazy markers).
- `stabbing_query(q)`: όλα τα διαστήματα που περιέχουν το σημείο q .
- `overlap_query([a,b])`: όλα τα διαστήματα που επικαλύπτονται με $[a,b]$.

Χρονική και χωρική πολυπλοκότητα.

- Έστω n ο αριθμός των διαστημάτων εισόδου και αφού $M \leq 4n + 1$ έχουμε ότι $M = O(n)$ ο αριθμός των elementary intervals. [BCK+08]
- `build(S)` συλλογή & ταξινόμηση άκρων: $O(n \log n)$, κατασκευή κόμβων: $O(n)$. [BCK+08]
- `insert([l,r])`: $O(\log n)$ (κάθε διάστημα αποθηκεύεται σε $O(\log n)$ κόμβους). [BCK+08]
- `delete([l,r])`: $O(\log n)$ με κατάλληλα handles ή επιπλέον δομές διαχείρισης. [BCK+08]
- `stabbing_query(q)` / `overlap_query([a,b])`: $O(\log n + k)$, όπου k ο αριθμός των αναφερόμενων διαστημάτων. [BCK+08]
- Χώρος: συνολικά $O(n \log n)$. [BCK+08; BF79]

Σημειώσεις.

- *Ευελιξία για aggregates*: αυτή η παραλλαγή είναι κατάλληλη όταν χρειάζεστε και reporting και range-aggregates (σε αντίθεση με centered interval trees που δεν υποστηρίζουν άμεσα aggregates). [BCK+08]
- *Δυναμικές διαγραφές*: απαιτούν handles ή δομημένες λίστες για αποδοτική αφαίρεση· χωρίς αυτά προτιμώνται lazy markers και περιοδική καθαριότητα. [BCK+08]
- *Γενικεύσεις*: η μέθοδος επεκτείνεται σε πολυδιάστατα προβλήματα ως *segment-tree of trees*, όμως το κόστος χώρου και χρόνου πολλαπλασιάζεται (συνχά οδηγεί σε $O(n \log^d n)$ χώρο/χρόνο για d διαστάσεις). [BCK+08]

5.3 Kd-tree

Ορισμός και υλοποίηση: Το Kd-tree είναι μια δομή διαμερισμού χώρου (*space-partitioning*) για σημεία σε k διαστάσεις: κάθε κόμβος ορίζει ένα υπερεπίπεδο (*hyperplane*) κάθετο σε κάποιον άξονα, το οποίο χωρίζει τον χώρο σε δύο ημιχώρους, έτσι ώστε τα σημεία του αριστερού (αντίστοιχα δεξιού) υποδέντρου να βρίσκονται στη μία (αντίστοιχα στην άλλη) πλευρά του split.

Στην κλασική μορφή, ο κόμβος αποθηκεύει και ένα σημείο-κλειδί, ενώ σε μια συχνή πρακτική παραλλαγή τα σημεία αποθηκεύονται μόνο στα φύλλα: οι εσωτερικοί κόμβοι κρατούν αποκλειστικά τη διάσταση και τη συντεταγμένη του split και τα φύλλα συγκεντρώνουν μικρούς κάδους (*buckets*) σημείων, κάτι που διευκολύνει την ομαδοποίηση και την αποδοτική χρήση cache/blocks (ιδίως σε I/O-aware υλοποιήσεις). [BF79; BCK+08; Sha78]

Η επιλογή της διάστασης του split γίνεται συχνά με κυκλική εναλλαγή (*cyclic splitting*), αλλά χρησιμοποιούνται και στρατηγικές όπως επιλογή της διάστασης με τη μεγαλύτερη διασπορά ή *median-based splits*. [BCK+08] Τα Kd-trees γενικεύουν την ιδέα των δυαδικών δέντρων αναζήτησης σε πολλαπλές διαστάσεις (σε 1 διάσταση συμπίπτουν με BST) και χρησιμοποιούνται ευρέως για *nearest-neighbour* αναζητήσεις, *orthogonal range queries*, χωρική δεικτοδότηση και collision detection. [BF79; BCK+08]

Σε δυναμικά σενάρια, απλές ενημερώσεις μπορεί να οδηγήσουν σε ανισορροπία, οπότε εφαρμόζονται *lazy rebuilding* τεχνικές όπου ένα «προβληματικό» υποδέντρο ξαναχτίζεται σε $O(m)$ χρόνο και το κόστος κατανέμεται *amortized* στις προηγούμενες ενημερώσεις. [BCK+08]

Υποστηριζόμενες λειτουργίες (operations).

- `build(P)`: κατασκευή ισορροπημένου Kd-tree από σύνολο σημείων P .
- `search(point)`: έλεγχος αν υπάρχει ακριβές σημείο.
- `insert(point)`: εισαγωγή νέου σημείου (δυναμική).
- `delete(point)`: διαγραφή σημείου (δυναμική — απαιτεί προσεκτική αντιμετώπιση για να διατηρηθεί η δομή).
- `range_query(rectangle)`: αναζήτηση/αναφορά όλων των σημείων εντός ορθογωνικής περιοχής (*orthogonal range search*).
- `k_nearest_neighbors(q,k)`: αλγόριθμος k-NN που επιστρέφει τα k εγγύτερα σημεία προς το ερώτημα q .

Πολυπλοκότητα λειτουργιών.

- `build` (ισορροπημένο, median-split): $O(n \log n)$ σε τυπικές υλοποιήσεις. [BCK+08; BF79]
- `search/insert/delete`: αποδοτικό κόστος $O(\log n)$ για καλά ισορροπημένα δέντρα· σε χειρότερες περιπτώσεις το ύψος και οι χρόνοι μπορούν να γίνουν $O(n)$. [BCK+08]
- `range_query`: για d -διάστατες ορθογωνικές (*orthogonal*) αναζητήσεις, το κόστος reporting είναι τυπικά υπο-γραμμικό ως προς n και της μορφής $O(n^{1-1/d} + k)$ (με κατάλληλες παραδοχές/παραλλαγές). [BCK+08; BF79]
- k-NN: σε χαμηλές διαστάσεις τα kd-trees είναι αποτελεσματικά με pruning, όμως το worst-case παραμένει $O(n)$ και η απόδοση υποβαθμίζεται καθώς αυξάνεται η διάσταση (*curse of dimensionality*). [BCK+08; BF79]

Σημειώσεις.

- *Curse of dimensionality*: σε υψηλές διαστάσεις η ικανότητα pruning του kd-tree αποδυναμώνεται και πολλά branches πρέπει να ελεγχθούν, οπότε η απόδοση τείνει προς γραμμική. [BCK+08]
- *Δυναμική χρήση*: απλές insert/delete οδηγούν σε απορρύθμιση· για συνεπή απόδοση απαιτούνται περιοδικές αναδόμησης ή ειδικές balanced παραλλαγές. [BCK+08]

5.4 Range Trees

Ορισμός και υλοποίηση: Ένα *Range Tree* είναι μια πολυεπίπεδη (multi-level) δομή δεδομένων για *orthogonal range searching*: δοθέντος ενός συνόλου σημείων $P \subseteq \mathbb{R}^d$ και ενός ορθογώνιου παραλληλεπιπέδου $R = [a_1, b_1] \times \dots \times [a_d, b_d]$, στόχος είναι να αναφερθούν (ή να μετρηθούν) όλα τα σημεία του P που ανήκουν στο R . Η δομή γενικεύει την ιδέα των μονοδιάστατων δέντρων αναζήτησης σε σταθερές διαστάσεις, με κύριο πλεονέκτημα τον μικρό χρόνο ερωτήματος εις βάρος αυξημένης μνήμης. [BF79; BCK+08]

Στην τυπική 2-διάστατη περίπτωση, κατασκευάζουμε ένα ισοζυγισμένο BST ως προς την x -συντεταγμένη (*primary tree*). Για κάθε κόμβο v του *primary tree*, ορίζουμε το υποσύνολο $P(v)$ των σημείων στο υποδέντρο του v και κατασκευάζουμε ένα *associated tree*, δηλαδή ένα δευτερεύον ισοζυγισμένο BST ως προς την y -συντεταγμένη, που αποθηκεύει ακριβώς τα σημεία του $P(v)$. Έτσι, ένα ορθογώνιο ερώτημα $[x_1, x_2] \times [y_1, y_2]$ απαντάται σε δύο στάδια: (1) στο *primary tree* εντοπίζουμε το x -εύρος και το αποσυνθέτουμε σε $O(\log n)$ κανονικούς κόμβους (*canonical nodes*) των οποίων τα υποδέντρα καλύπτονται πλήρως από το $[x_1, x_2]$, και (2) για κάθε τέτοιο κόμβο v εκτελούμε ένα *range query* ως προς y στο *associated tree* του v και αναφέρουμε τα αντίστοιχα σημεία. Με αυτήν τη διάταξη, ο χρόνος ερωτήματος είναι τυπικά $O(\log^2 n + k)$, όπου k το πλήθος των σημείων που αναφέρονται, ενώ ο χώρος είναι $O(n \log n)$ επειδή κάθε σημείο αποθηκεύεται σε *associated trees* για $O(\log n)$ κόμβους κατά μήκος της διαδρομής του στο *primary tree*. [BCK+08]

Η ιδέα γενικεύεται σε $d > 2$ διαστάσεις προσθέτοντας επίπεδα: σε κάθε κόμβο του *primary tree* αποθηκεύεται μια $(d-1)$ -διάστατη *range* δομή για τα σημεία του $P(v)$, με τυπικές εγγυήσεις χώρου $O(n \log^{d-1} n)$ και χρόνου ερωτήματος $O(\log^d n + k)$ για σταθερό d . [BCK+08]

Λειτουργίες που υποστηρίζονται

- **Build(S):** κατασκευή της δομής από στατικό σύνολο n σημείων. [BCK+08]
- **Range Query($[x, x']$, $[y, y']$):** εντοπισμός *split node* στο κύριο δέντρο και επιλογή $O(\log n)$ *canonical* υποδέντρων, όπου για καθένα εκτελείται αναζήτηση στο βοηθητικό του (ως προς y). [BCK+08]
- **Range Counting($[x, x']$, $[y, y']$):** καταμέτρηση χωρίς reporting, με χρήση μεγεθών υποδέντρων/κατάλληλων counters στις βοηθητικές δομές. [BCK+08]
- **Insert/Delete(p):** σε δυναμικές εκδοχές, απαιτεί ενημέρωση $O(\log n)$ βοηθητικών δομών και μηχανισμό διατήρησης ισορροπίας (π.χ. weight-balanced παραλλαγές), συνήθως με amortized κόστος. [BCK+08]

Χρονική και Χωρική πολυπλοκότητα

- **Χωρική Πολυπλοκότητα:** $O(n \log n)$ για 2 διαστάσεις. [BCK+08; BF79]
- **Χρόνος Κατασκευής:** $O(n \log n)$. [BCK+08]
- **Χρόνος Range Query:** $O(\log^2 n + k)$ (2D), όπου k το πλήθος των αποτελεσμάτων. [BCK+08]
- **Χρόνος Range Counting:** $O(\log^2 n)$ (2D). [BCK+08]
- **Χρόνος Εισαγωγής/Διαγραφής:** τυπικά $O(\log^2 n)$ (amortized) σε δυναμικές παραλλαγές. [BCK+08]

Σημειώσεις

- *Fractional Cascading (Κλασματική Διαδοχή)*: αντικατάσταση των δευτερευουσών δομών από διασυνδεδεμένους ταξινομημένους πίνακες ώστε οι $O(\log n)$ αναζητήσεις σε βοηθητικές δομές να «μοιράζονται» μία δυαδική αναζήτηση. Έτσι, σε τυπικές 2D περιπτώσεις ο χρόνος query μπορεί να μειωθεί σε $O(\log n + k)$. [CG86; BCK+08]
- *Γενίκευση σε d διαστάσεις*: αναδρομικός ορισμός με κύριο δέντρο στην πρώτη διάσταση και βοηθητικά Range Trees $d - 1$ διαστάσεων. Τότε ο χώρος γίνεται $O(n \log^{d-1} n)$ και ο χρόνος query $O(\log^d n + k)$. [BCK+08; BF79]

6 Επιλεγμένες Προηγμένες Δομές Δεδομένων

Σε αυτήν την ενότητα εξετάζουμε ειδικές δομές δεδομένων που δεν εντάσσονται άμεσα στις προηγούμενες κατηγορίες, αλλά λειτουργούν ως βασικά δομικά συστατικά στον σχεδιασμό πιο σύνθετων κατασκευών, συχνά ως βοηθητικές (*auxiliary*) δομές.

Συγκεκριμένα, τόσο η *Priority Queue with Attrition* (PQA) όσο και ο *Q-heap* αξιοποιούνται από τους Brodal και Tsakalidis στην κατασκευή βέλτιστης δομής για το πρόβλημα *dynamic planar range skyline reporting*. [BT11; FW93] Η απαίτηση για ισχυρότερη μορφή της PQA (ιδίως ως προς τη δυνατότητα σύνδεσης/συνένωσης δομών) οδηγεί στη *Catenable Priority Queue with Attrition* (CPQA), η οποία προκύπτει φυσικά και στη σχεδίαση δομών για το ίδιο πρόβλημα στο μοντέλο εξωτερικής μνήμης (I/O). [KTT+13; AV88]

6.1 Priority Queue with Attrition

Η *Priority Queue with Attrition* (PQA) είναι παραλλαγή της κλασικής *priority queue* για πλήρως διατεταγμένα κλειδιά, η οποία επιτρέπει την λειτουργία *attrition*, δηλαδή τη μόνιμη απόρριψη στοιχείων σύμφωνα με κανόνες που επιβάλλονται από τις διαδέσιμες λειτουργίες της δομής. [Sun89]

Σε θεωρητικό επίπεδο, η PQA ορίζεται μέσω ενός περιορισμένου συνόλου λειτουργιών (βλ. παρακάτω) και χρησιμοποιείται ως βοηθητική (*auxiliary*) δομή σε κατασκευές όπου θέλουμε να διατηρούμε μόνο τα στοιχεία που μπορούν να επηρεάσουν μελλοντικά ερωτήματα. Το *attrition* επιτρέπει την άμεση αποκοπή «μη-χρήσιμων» στοιχείων, μειώνοντας τον όγκο δεδομένων που χρειάζεται να ενημερώνεται ή να εξετάζεται, κάτι ιδιαίτερα χρήσιμο σε σχήματα τύπου *sweep-line* και σε εφαρμογές *skyline/range-reporting*. [Sun89]

Λειτουργίες. Οι βασικές λειτουργίες της PQA, όπως ορίζονται από τον Sundar, είναι οι εξής:

- **CREATEPQA():** δημιουργία και αρχικοποίηση κενής PQA.
- **DELETEMIN():** διαγραφή και επιστροφή του ελάχιστου στοιχείου.
- **INSERT(x):** εισαγωγή του x και ταυτόχρονη *attrition*, δηλαδή διαγραφή όλων των στοιχείων $y \in S$ με $y \geq x$.

Συχνά θεωρούνται επίσης χρήσιμες οι παρακάτω βοηθητικές λειτουργίες:

- **FINDMIN():** επιστροφή του ελάχιστου χωρίς διαγραφή (π.χ. αν το *min* διατηρείται ρητά).
- **DELETE(x):** διαγραφή συγκεκριμένου στοιχείου x , υπό την προϋπόθεση ότι διαθέτουμε δείκτη/αναφορά στη θέση του μέσα στη δομή.

Πολυπλοκότητα λειτουργιών. Ο Sundar παρουσιάζει αρχικά μια απλή υλοποίηση με $O(1)$ επιμερισμένο (*amortized*) κόστος ανά βασική πράξη και στη συνέχεια περιγράφει τεχνικές απο-επιμερισμού (*de-amortization*) που δίνουν εγγυημένο $O(1)$ χρόνο χειρότερης περίπτωσης για τις **CREATEPQA()**, **DELETEMIN()** και **INSERT(x)**. [Sun89]

Η μετάβαση από *amortized* σε *worst-case* επιτυγχάνεται με ελεγχόμενα *clean-up* σχήματα (*forward* και *backward cleaning*), διατήρηση μικρών βοηθητικών *buffers* και διαμοιρασμό του κόστους αναδόμησης σε πολλαπλές κλήσεις ώστε κάθε μεμονωμένη κλήση να εκτελεί μόνο σταθερή εργασία. Ο Sundar τεκμηριώνει τα *invariants* και αποδεικνύει ότι υπό αυτές τις τεχνικές οι λειτουργίες εκτελούνται σε $O(1)$ *worst-case*. [Sun89]

Υλοποίηση Στο **Κεφάλαιο 9 (Υλοποίηση)** και σε ξεχωριστά αρχεία κώδικα παρέχουμε υλοποιήσεις τόσο της amortized δομής όσο και της worst case δομής (και με τα δύο clean up σχήματα) τόσο σε python όσο και σε C++.

Σημείωση 1: Μη ύπαρξη Lower Bound για Priority Queue with Attrition.

Μια Priority Queue with Attrition (PQA) επιτυγχάνει $O(1)$ για τις βασικές πράξεις στη χειρότερη περίπτωση, επειδή δεν επιχειρεί ποτέ να διατηρήσει ή να επανακατασκευάσει μια πλήρη, συγκριτικά ταξινομημένη διάταξη των στοιχείων: ο μηχανισμός *attrition* απορρίπτει «κατεχόμενα/υποδεέστερα» στοιχεία κατά την εισαγωγή και οι δομικές αναλλοίωτες (bias condition, sublist invariants) εξασφαλίζουν ότι η τιμή ερωτήματος (π.χ. το $\min$) είναι άμεσα διαθέσιμη ή υπολογίζεται με σταθερή τοπική εργασία, ακόμη και αν η πλήρης διάταξη δεν υφίσταται. [Sun89] Επομένως, η PQA δεν λύνει προβλήματα γενικής ταξινόμησης ή γενικής αναζήτησης διαδοχικού στοιχείου σε πλήρη σειρά και έτσι δεν υπάγεται στο συγκριτικό κατώτατο όριο του sorting / nearest-neighbour.

Σημείωση 2: Ύπαρξη Lower Bound για Priority Stack with Attrition.

Η Priority Stack with Attrition (PSA) είναι μια δομή δεδομένων της οποίας η μόνη διαφορά από την PQA είναι ότι η λειτουργία Insert τώρα διαγράφει τα στοιχεία που είναι μικρότερα ή ίσα με το εισαγόμενο στοιχείο. Σε αντίθεση με την PQA, η PSA δεν μπορεί να υλοποιηθεί σε $O(1)$ worst-case: υπάρχει σαφής αναγωγή του προβλήματος του «επόμενου μεγαλύτερου στοιχείου» (nearest/successor query) σε πράξεις PSA. Μέσω της αναγωγής αυτής μπορούμε να δείξουμε ότι κάθε δομή δεδομένων για την PSA θα πρέπει να ξοδέψει $\Omega(\log n)$ χρόνο είτε για τη λειτουργία Insert είτε για τη λειτουργία DeleteMin, αλλιώς θα υπήρχε τρόπος να παρακάμψουμε γνωστά δυναμικά lower bounds για predecessor/successor στο cell-probe μοντέλο. [FS89; PD06]

Συνεπώς, το $\Omega(\log n)$ bound είναι αναγκαίο και η PSA **δεν** έχει $O(1)$ worst-case υλοποίηση, ενώ η PQA το αποφεύγει επειδή δεν μπορεί να χρησιμοποιηθεί για ταξινόμηση.

6.2 Catenable Priority Queue with Attrition

Η I/O-Catenable Priority Queue with Attrition (I/O-CPQA) επεκτείνει την Priority Queue with Attrition στο μοντέλο εξωτερικής μνήμης (I/O model), προσθέτοντας τη δυνατότητα *catenation* δύο δομών με ταυτόχρονη εφαρμογή *attrition*. Στόχος είναι η ελαχιστοποίηση των μεταφορών blocks μεταξύ κύριας μνήμης και δίσκου, σε αντίθεση με υλοποιήσεις σε pointer-machine όπου το κόστος μετριέται σε βασικές πράξεις δείκτη. [KTT+13; AV88]

Λειτουργίες. Μια I/O-CPQA που αποθηκεύει ένα σύνολο στοιχείων με ολική διάταξη υποστηρίζει τις παρακάτω πράξεις: [KTT+13]

- FINDMIN(Q): επιστρέφει το $\min(Q)$ χωρίς να τροποποιεί τη δομή.
- DELETETEMIN(Q): επιστρέφει και αφαιρεί το $\min(Q)$ (η πράξη είναι *destructive* ως προς το Q).
- CATENATEANDATTRITE(Q_1, Q_2): (με προϋπόθεση $Q_2 \neq \emptyset$) επιστρέφει μια νέα δομή

$$Q' = \{e \in Q_1 \mid e < \min(Q_2)\} \cup Q_2,$$

δηλαδή ενώνει τις δύο δομές και ταυτόχρονα αφαιρεί από το Q_1 όλα τα στοιχεία $e \geq \min(Q_2)$. Μετά την πράξη, τα Q_1 και Q_2 δεν είναι πλέον διαθέσιμα (destructive operation).

Πολυπλοκότητα λειτουργιών. Όσον αφορά την πολυπλοκότητα, η δομή επιτυγχάνει $O(1)$ I/Os στη χειρότερη περίπτωση για όλες τις βασικές λειτουργίες. Επιπλέον, υποστηρίζει επιμερισμένο (amortized) κόστος $O(1/b)$ I/Os, όπου B είναι το μέγεθος του μπλοκ δίσκου και $1 \leq b \leq B$, υπό την προϋπόθεση ότι ένας σταθερός αριθμός από κρίσιμα δεδομένα (critical records) είναι προ-φορτωμένα (preloaded) στην κύρια μνήμη. [KTT+13; AV88]

6.3 Q-Heap

Το *Q-Heap* αποτελεί μια εξειδικευμένη δομή δεδομένων που λειτουργεί στο μοντέλο RAM με μέγεθος λέξης (word size) w , όπου ισχύει $w \geq \log n$. [FW93] Ο βασικός σκοπός της δομής είναι η αποθήκευση ενός μικρού υποσυνόλου ακεραίων αριθμών από το σύμπαν $U = \{0, \dots, 2^w - 1\}$. Συγκεκριμένα, το Q-Heap έχει χωρητικότητα που περιορίζεται αυστηρά σε το πολύ $\log^{1/4} n$ στοιχεία. Η λειτουργία του βασίζεται στην ύπαρξη ενός καθολικού, προ-υπολογισμένου πίνακα αναζήτησης (global precomputed lookup-table), ο οποίος επιτρέπει την εκμετάλλευση του παραλληλισμού σε επίπεδο bit (word-level parallelism) για την ταχεία εκτέλεση πράξεων. [FW93]

Λειτουργίες. Οι βασικές λειτουργίες που υποστηρίζει ο Q-Heap είναι οι εξής: [FW93]

- INSERT(x) : Εισαγωγή του ακεραίου x στο σύνολο στοιχείων της δομής.
- DELETE(x) : Διαγραφή του ακεραίου x από τη δομή.
- PREDECESSOR(x) : Αναζήτηση και επιστροφή του αμέσως μικρότερου στοιχείου (ή του ίδιου του x αν υπάρχει) που βρίσκεται αποθηκευμένο στη δομή. Αυτή η λειτουργία καλύπτει και την έννοια της γενικής αναζήτησης (search).

Πολυπλοκότητα λειτουργιών. Όσο αφορά την χρονική πολυπλοκότητα των λειτουργιών, το Q-Heap επιτυγχάνει εξαιρετικά αποδοτικά φράγματα. Όλες οι παραπάνω λειτουργίες (INSERT, DELETE, PREDECESSOR) εκτελούνται σε σταθερό χρόνο $O(1)$ στη χειρότερη περίπτωση (worst-case). Η ίδια η δομή απαιτεί χώρο $O(\log^{1/4} n)$. [FW93] Ωστόσο, για να επιτευχθούν αυτοί οι χρόνοι, απαιτείται η διαδεσιμότητα του καθολικού πίνακα αναζήτησης, ο οποίος χρειάζεται $O(n)$ χρόνο προ-επεξεργασίας (preprocessing) και καταλαμβάνει $O(n)$ χώρο μνήμης. [FW93]

Σημείωση. Συνεπώς, το Q-Heap είναι ιδανικό ως δομικό στοιχείο (building block) σε μεγαλύτερες δομές όπου το κόστος της προ-επεξεργασίας μπορεί να αποσβεστεί.

7 External Memory και Buffering Framework

Σε αυτήν την ενότητα παρουσιάζονται οι βασικές αρχές σχεδίασης δομών δεδομένων στο μοντέλο εξωτερικής μνήμης (I/O model) και ο τρόπος με τον οποίο το κόστος μεταφοράς blocks καθορίζει την επιλογή των κατάλληλων δομών. Ξεκινάμε από τα *B-trees* ως την κλασική λύση για αποδοτική αναζήτηση και ενημέρωση στη μνήμη (σκληρό δίσκο) [BM72]. Στη συνέχεια, εξετάζουμε *παραλλαγές των B-trees* που προκύπτουν είτε από αλλαγές στην οργάνωση των φύλλων/δεικτών (όπως τα *B+-trees*) είτε από την ανάγκη για βελτιστοποίηση των εγγραφών (write-optimization) όπως τα *B^e-trees* [BFJ+15].

Έπειτα, εισάγεται το *buffering framework* του Arge [Arg03]: μια γενική τεχνική που «ενισχύει» tree-based internal-memory δομές με buffers, ώστε οι ενημερώσεις να εκτελούνται σε παρτίδες (batched) με τεμπέλικό (lazy) τρόπο, επιτυγχάνοντας βελτιωμένα αποσβεσμένα (amortized) I/O φράγματα.

Τέλος, παρουσιάζεται ένα συγκεκριμένο παράδειγμα εφαρμογής της ιδέας: η μετατροπή ενός (left-leaning) red-black tree [Sed08] σε buffered εκδοχή. Αξιοποιώντας τη γνωστή αντιστοιχία $\text{LLRB} \leftrightarrow 2\text{--}3\text{--}4 \text{ trees} \leftrightarrow (a, b)\text{-trees}$, δείχνουμε πώς ένα ισορροπημένο BST εντάσσεται φυσικά στο πολυδενδρικό (multiway) πλαίσιο της εξωτερικής μνήμης.

7.1 B-trees

Ορισμός και περιγραφή. Τα B-trees ορίστηκαν από τους R. Bayer και E. McCreight το 1972 [BM72] ως μια γενική δομή δεδομένων για την αποτελεσματική αποθήκευση και ανάκτηση μεγάλων συλλογών κλειδιών σε εξωτερική μνήμη, ελαχιστοποιώντας τον αριθμό των προσπελάσεων στο δίσκο (disk accesses).

Ένα *B-tree* τάξης m (όπου m είναι ο μέγιστος αριθμός παιδιών) ικανοποιεί τα εξής invariants: [Knu98]

- Κάθε κόμβος έχει το πολύ m παιδιά.
- Κάθε κόμβος, εκτός από τη ρίζα και τα φύλλα, έχει τουλάχιστον $\lceil m/2 \rceil$ παιδιά.
- Η ρίζα έχει τουλάχιστον 2 παιδιά, εκτός αν είναι φύλλο.
- Όλα τα φύλλα βρίσκονται στο ίδιο επίπεδο.
- Κάθε μη-φύλλο με k παιδιά περιέχει ακριβώς $k-1$ κλειδιά, σε αύξουσα σειρά. Τα κλειδιά αυτά λειτουργούν ως διαχωριστές των k υποδέντρων: αν είναι $a_1 < \dots < a_{k-1}$ και τα παιδιά $c_0, \dots, c_{k-1}$, τότε όλα τα κλειδιά στο c_0 είναι $< a_1$, στο c_i (για $1 \leq i \leq k-2$) είναι μεταξύ a_i και a_{i+1} , και στο c_{k-1} είναι $> a_{k-1}$.
- Στο I/O μοντέλο επιλέγουμε την τάξη m έτσι ώστε ένας κόμβος (με τα κλειδιά και τους δείκτες του) να χωρά σε ένα block δίσκου μεγέθους B : συνεπώς $m = \Theta(B)$ και μία επίσκεψη σε κόμβο κοστίζει $O(1)$ I/Os.

Λειτουργίες.

- SEARCH(x): αναζήτηση του x και επιστροφή θέσης/αναφοράς (ή αποτυχία).
- INSERT(x): εισαγωγή του x διατηρώντας τα invariants (με split σε περίπτωση υπερχειλίσας κόμβου).

- **DELETE(x)**: διαγραφή του x διατηρώντας τα invariants (με borrow/transfer ή merge/fusion σε περίπτωση υποπλήρωσης).
- **MIN()**/**MAX()**: επιστροφή ελάχιστου/μέγιστου κλειδιού.
- **SUCCESSOR(x)**/**PREDECESSOR(x)**: επόμενο/προηγούμενο κλειδί ως προς τη διάταξη (όπου ορίζεται).

Πολυπλοκότητα λειτουργιών.

- **Ύψος**. Για n κλειδιά, το ύψος είναι $h = O(\log_m n)$ (όπου m η τάξη/μέγιστος αριθμός παιδιών).
- **Κόστος I/O**. Αν κάθε κόμβος χωρά σε ένα block μεγέθους B και επιλέγουμε $m = \Theta(B)$, τότε κάθε βασική πράξη κοστίζει $O(\log_B N)$ I/Os, όπου N το πλήθος κλειδιών στο δίσκο.

Σημειώσεις.

- Στο INSERT, ένα split μπορεί να διαδοθεί προς τη ρίζα· αν η ρίζα διασπαστεί, δημιουργείται νέα ρίζα και το ύψος αυξάνει κατά 1.
- Στο DELETE, αν ένας κόμβος πέσει κάτω από το ελάχιστο επιτρεπτό πλήθος παιδιών/κλειδιών, αποκαθίσταται είτε με borrow/transfer από αδελφό είτε με merge/fusion (με μεταφορά κατάλληλου separator από/προς τον γονέα).
- Ο συμβολισμός διαφέρει στη βιβλιογραφία: στον Knuth η τάξη m είναι το μέγιστο πλήθος παιδιών, άρα (εκτός ρίζας) $\# \text{παιδιών} \in [\lceil m/2 \rceil, m]$, ενώ αλλού η παράμετρος (συχνά επίσης m ή t) είναι το ελάχιστο πλήθος παιδιών, άρα $\# \text{παιδιών} \in [m, 2m]$. [Knu98; CLR+09]

7.2 Παραλλαγές των B-Trees

Με την πάροδο του χρόνου αναπτύχθηκαν παραλλαγές για την εξυπηρέτηση ειδικών απαιτήσεων:

- **B+ tree**: Η πιο διαδεδομένη παραλλαγή σε συστήματα αρχείων και βάσεις δεδομένων [Com79]. Οι εσωτερικοί κόμβοι αποθηκεύουν μόνο κλειδιά-οδηγούς (routing keys) και όχι δεδομένα. Όλα τα δεδομένα (ή οι δείκτες προς τις εγγραφές) βρίσκονται στα φύλλα, τα οποία συνδέονται μεταξύ τους με λίστα (linked list). *Πλεονέκτημα*: Τα range queries εκτελούνται πολύ αποδοτικά με μία γραμμική σάρωση των φύλλων (sequential scan), ενώ οι εσωτερικοί κόμβοι έχουν μεγαλύτερο fan-out.
- **B* tree**: Στοχεύει στην καλύτερη αξιοποίηση του χώρου. Απαιτεί οι κόμβοι να είναι τουλάχιστον $2/3$ γεμάτοι (αντί για $1/2$). Για να το πετύχει αυτό, καθυστερεί τη διάσπαση (split) ενός κόμβου διαμοιράζοντας τα κλειδιά με έναν αδελφό κόμβο. Μόνο όταν και οι δύο γεμίσουν, διασπώνται σε τρεις κόμβους (two-to-three split) [BM72].
- **B⁻ tree**: Μια σύγχρονη παραλλαγή βελτιστοποιημένη για εγγραφές (write-optimized) [BF+15]. Χρησιμοποιεί εσωτερικούς buffers στους κόμβους για να συσσωρεύει τις ενημερώσεις και να τις προωθεί μαζικά στα παιδιά, θυσιάζοντας ένα μικρό μέρος της ταχύτητας ανάγνωσης (point queries) για δραματική αύξηση της ταχύτητας εγγραφών (inserts/updates). Αποτελεί πρόδρομο των πλήρως buffered δομών.
- **Order-statistic B-tree**: Εμπλουτίζει κάθε κόμβο με έναν μετρητή (subtree size), επιτρέποντας την εκτέλεση ερωτημάτων *Select* (βρες το i -οστό στοιχείο) και *Rank* (βρες την κατάταξη του στοιχείου x) σε χρόνο $O(\log_B n)$ [CLR+09].

7.3 Buffer tree

Ορισμός, περιγραφή και υλοποίηση. Το *Buffer Tree*, το οποίο εισήχθη από τον Lars Arge, είναι μια δομή δεδομένων εξωτερικής μνήμης (external memory data structure) σχεδιασμένη για την αποδοτική εκτέλεση *ομαδοποιημένων* (batched) λειτουργιών ενημέρωσης και ερωτημάτων [Arg03]. Σε αντίθεση με τα κλασικά B-trees που βελτιστοποιούν μεμονωμένες (online) λειτουργίες, το Buffer Tree στοχεύει στη μεγιστοποίηση της απόδοσης όταν έχουμε να διαχειριστούμε μεγάλες ροές δεδομένων. Η βασική ιδέα είναι η εκμετάλλευση της εσωτερικής μνήμης M για την καθυστέρηση (*lazy evaluation*) των λειτουργιών, συσσωρεύοντάς τις σε buffers και εφαρμόζοντάς τις μαζικά προς τα φύλλα μόνο όταν είναι απαραίτητο. Με αυτόν τον τρόπο, το κόστος κάθε μεταφοράς δεδομένων (I/O) επιμερίζεται σε πολλά στοιχεία.

Η δομή βασίζεται σε ένα δέντρο αναζήτησης πολύ μεγάλης διακλάδωσης (multi-way search tree), παρόμοιο με το B-tree, με τις εξής δομικές προδιαγραφές:

- *Διακλάδωση (Fan-out):* Κάθε εσωτερικός κόμβος v έχει βαθμό διακλάδωσης $m = \Theta(M/B)$, δηλαδή έχει πλήθος παιδιών ανάλογο του πόσα blocks χωράνε στη μνήμη.
- *Buffers:* Το κρίσιμο χαρακτηριστικό είναι ότι κάθε εσωτερικός κόμβος διαθέτει έναν *buffer* μεγέδους $\Theta(M)$ (συνήθως $M/2$). Ο buffer αυτός αποθηκεύει προσωρινά τις εντολές (insertions, deletions, updates) που "κατεβαίνουν" από τη ρίζα.
- *Φύλλα:* Τα φύλλα περιέχουν τα πραγματικά δεδομένα (ή υποδέντρα αν η δομή χρησιμοποιείται ως base structure) και όταν γεμίσουν, υπόκεινται σε split όπως στα B-trees.
- *Invariants:* Όλα τα φύλλα βρίσκονται στο ίδιο επίπεδο και οι λειτουργίες εξισορρόπησης είναι παρόμοιες με των (a, b) -trees, προσαρμοσμένες ώστε να λαμβάνουν υπόψη το άδειασμα των buffers.

Υποστηριζόμενες Λειτουργίες. Οι λειτουργίες στο Buffer Tree δεν εκτελούνται άμεσα, αλλά "σημειώνονται" με χρονοσφραγίδα (timestamp) για να διατηρηθεί η χρονική τους αλληλουχία:

- *Update(Insert/Delete):* Η λειτουργία προστίθεται στον buffer της ρίζας. Δεν κατεβαίνουμε στο δέντρο. Αν ο buffer γεμίσει, εκτελείται διαδικασία *Buffer Emptying*.
- *Buffer Emptying(Flush):* Όταν ο buffer ενός κόμβου γεμίσει, τα στοιχεία του ταξινομούνται στην κύρια μνήμη και διανέμονται (push down) στους buffers των παιδιών του. Αν κάποιος buffer παιδιού γεμίσει, η διαδικασία συνεχίζεται αναδρομικά.
- *Range Query(Batched):* Τα ερωτήματα εύρους αντιμετωπίζονται επίσης ως "εγγραφές" που ρέουν προς τα κάτω. Όταν ένα ερώτημα φτάσει στα φύλλα, απαντάται με βάση τα δεδομένα εκεί. Τα αποτελέσματα μπορούν να συλλεχθούν στο τέλος.
- *Deletemin:* Ειδική λειτουργία για χρήση ως Priority Queue. Απαιτεί το άδειασμα των buffers κατά μήκος του αριστερότερου μονοπατιού [Arg03].

Πολυπλοκότητα I/O. Έστω N το πλήθος των στοιχείων, M το μέγεθος της μνήμης και B το μέγεθος του block. Το κύριο πλεονέκτημα του Buffer Tree είναι το εξαιρετικά χαμηλό *Amortized Update Cost*. Κάθε ενημέρωση κοστίζει:

$$O\left(\frac{1}{B} \log_{M/B} \frac{N}{B}\right) \text{ I/Os}$$

Αυτό σημαίνει ότι για κάθε I/O πράξη, επεξεργαζόμαστε B στοιχεία (σε αντίθεση με το 1 στοιχείο ανά I/O του B-tree στη χειρότερη περίπτωση). Η χρήση Buffer Tree για την εισαγωγή N στοιχείων και την ανάγνωσή τους (in-order traversal) ισοδυναμεί με το βέλτιστο φράγμα ταξινόμησης στην εξωτερική μνήμη.

Σημείωση. Τα Buffer Trees αποτελούν τη βάση για αποδοτικούς αλγορίθμους εξωτερικής μνήμης σε γράφους (π.χ. list ranking) και γεωμετρικά προβλήματα (π.χ. κατασκευή Segment Trees σε εξωτερική μνήμη μέσω της τεχνικής *batch filtering*).

7.4 Buffering Framework

Γενική Ιδέα και Μετασχηματισμός. Το Buffering Framework αποτελεί μια γενική τεχνική που εισήγαγε ο Lars Arge για τη μετατροπή δομών δεδομένων εσωτερικής μνήμης σε αποδοτικές δομές εξωτερικής μνήμης [Arg03]. Η διαδικασία ξεκινά με την ομαδοποίηση των κόμβων της αρχικής δομής σε «υπερ-κόμβους» (super-nodes). Συγκεκριμένα, οι κόμβοι ομαδοποιούνται έτσι ώστε κάθε υπερ-κόμβος να αποκτήσει βαθμό διακλάδωσης (fan-out) $\Theta(m)$, όπου $m = M/B$ είναι ο αριθμός των blocks που χωρούν στην κύρια μνήμη. Αυτός ο μετασχηματισμός μειώνει δραστικά το ύψος του δέντρου σε $O(\log_m n)$, επιτρέποντας τη βέλτιστη θεωρητική απόδοση.

Δομή και Buffers. Το βασικό χαρακτηριστικό της τεχνικής είναι ότι σε κάθε υπερ-κόμβο ανατίθεται ένας *buffer* μεγέθους $\Theta(M)$ (δηλαδή $\Theta(m)$ blocks). Οι λειτουργίες (όπως insert, delete, search) δεν εκτελούνται άμεσα διασχίζοντας το δέντρο. Αντίθετα, κάθε νέα εντολή εισάγεται αρχικά στον buffer της ρίζας. Μόνο όταν ένας buffer γεμίσει, εκτελείται η διαδικασία *buffer-emptying*. Κατά τη διαδικασία αυτή, τα στοιχεία του buffer ταξινομούνται στην εσωτερική μνήμη και διανέμονται (pushed down) στους buffers των παιδιών του υπερ-κόμβου.

Χρονική Αλληλουχία (Time Stamping). Επειδή οι λειτουργίες εκτελούνται ετεροχρονισμένα (lazy evaluation), είναι πιθανό να συνυπάρχουν στο δέντρο πολλαπλές εντολές για το ίδιο στοιχείο σε διαφορετικά επίπεδα (π.χ. μια εντολή διαγραφής που ακολουθεί χρονικά μια εντολή εισαγωγής). Για να διασφαλιστεί η ορθότητα, κάθε λειτουργία λαμβάνει μια *χρονοσφραγίδα* (time stamp) κατά την εισαγωγή της. Έτσι, όταν τα στοιχεία συναντηθούν κατά το άδειασμα των buffers, η χρονοσφραγίδα καθορίζει τη σειρά εφαρμογής τους, επιτρέποντας την ορθή αλληλεπίδραση (π.χ. ακύρωση μιας εισαγωγής από μια μεταγενέστερη διαγραφή).

Ανάλυση Κόστους. Η αποδοτικότητα της τεχνικής βασίζεται στο γεγονός ότι το άδειασμα ενός buffer μεγέθους M απαιτεί $O(m)$ I/Os, δηλαδή γραμμικό κόστος ως προς τα blocks του buffer. Επειδή η διαδικασία εκτελείται μόνο όταν ο buffer είναι γεμάτος, το κόστος μεταφοράς των στοιχείων στο επόμενο επίπεδο επιμερίζεται. Κάθε στοιχείο «χρεώνεται» μόνο $O(1/B)$ I/Os για κάθε επίπεδο που κατεβαίνει. Με ύψος δέντρου $O(\log_m n)$, το συνολικό επιμερισμένο (amortized) κόστος ανά λειτουργία διαμορφώνεται σε $O(\frac{1}{B} \log_m n)$ I/Os.

7.4.1 Μετατροπή Red Black Tree σε Buffered Red Black Tree: Απόδειξη

Στόχος και Κίνητρο. Ο βασικός στόχος αυτής της κατασκευής είναι η μετατροπή των κλασικών αυτο-ισορροπούμενων δυαδικών δέντρων αναζήτησης (Self-Balancing BSTs) σε μια *buffered* εκδοχή, κατάλληλη για το μοντέλο εξωτερικής μνήμης. Αν και η βιβλιογραφία θρίβει αναφορών για B-trees και Buffer Trees, η συγκεκριμένη προσαρμογή ενός Red-Black tree σε δομή εξωτερικής μνήμης αποτελεί συχνά «κοινή γνώση» (folklore) ή τεχνική άσκηση, χωρίς να αποδίδεται πάντα σε ένα μοναδικό citation ως ξεχωριστή δομή. Ωστόσο, η χρησιμότητά

της είναι προφανής: μας επιτρέπει να διατηρήσουμε τη λογική ενός δυαδικού δέντρου που γνωρίζουμε από την εσωτερική μνήμη, προσθέτοντας τα πλεονεκτήματα των buffers για τις I/O λειτουργίες.

Επιλογή Δομής: Left-Leaning Red-Black Tree. Για την υλοποίηση αυτή επιλέγουμε ως βάση το *Left-Leaning Red-Black (LLRB) Tree*, μια παραλλαγή των Red-Black trees που εισήχθη από τον Robert Sedgwick [Sed08]. Ένα LLRB tree είναι ένα δυαδικό δέντρο αναζήτησης με επιπλέον πληροφορία χρώματος (κόκκινο/μαύρο) στους συνδέσμους (links) ή στους κόμβους, το οποίο ικανοποιεί συγκεκριμένες συνθήκες ισορροπίας. Η βασική διαφορά του από τα κλασικά Red-Black trees είναι η επιβολή της συνθήκης ότι οι «κόκκινοι» σύνδεσμοι πρέπει να γέρνουν προς τα αριστερά (left-leaning). Αυτός ο περιορισμός, χωρίς να περιορίζει την εκφραστική δυνατότητα των Red-Black trees, μειώνει δραματικά τις περιπτώσεις που πρέπει να εξετάσουμε κατά την υλοποίηση, καθιστώντας τον κώδικα πιο συμπαγή και την ανάλυση πιο ξεκάθαρη.

Αλυσίδα ισομορφισμών. Η σχέση μεταξύ των δομών μπορεί να ιδωθεί ως η ακόλουθη αλυσίδα 1-1 αντιστοιχιών:

- **LLRB $\leftrightarrow$ 2-3 trees.** Σε ένα *Left-Leaning Red-Black* δέντρο οι κόκκινες ακμές επιτρέπονται μόνο προς τα αριστερά, άρα κάθε 3-κόμβος ενός 2-3 tree έχει μοναδική αναπαράσταση ως «μαύρος κόμβος με ένα αριστερό κόκκινο παιδί». Με αυτήν τη σύμβαση, κάθε 2-3 κόμβος αντιστοιχεί σε ένα μοναδικό μαύρο-κόκκινο σύμπλεγμα και αντίστροφα.
- **2-3 trees $\leftrightarrow$ B-trees τάξης 3.** Ένα 2-3 tree είναι ακριβώς ένα B-tree τάξης 3: κάθε κόμβος έχει 2 ή 3 παιδιά (δηλαδή περιέχει 1 ή 2 κλειδιά), και όλα τα φύλλα βρίσκονται στο ίδιο επίπεδο.

Σημείωση. Τα γενικά Red-Black trees (χωρίς τον left-leaning περιορισμό) αντιστοιχούν σε 2-3-4 trees, δηλαδή σε B-trees τάξης 4, επειδή μπορούν να αναπαραστήσουν και 4-κόμβους (3 κλειδιά / 4 παιδιά) μέσω «μαύρου κόμβου με δύο κόκκινα παιδιά».

Εφαρμογή του Buffering Framework. Έχοντας αναγάγει το Red-Black tree σε B-tree, μπορούμε πλέον να επικαλεστούμε την κατασκευή του *Buffer Tree* που περιγράψαμε στην προηγούμενη ενότητα [Arg03]. Το Buffering Framework του Arge εφαρμόζεται γενικά σε οποιαδήποτε δομή τύπου (a,b) -tree (και άρα σε B-trees). Με αυτόν τον τρόπο, οι αλγόριθμοι εισαγωγής και διαγραφής του LLRB (περιστροφές, αλλαγές χρωμάτων) εκτελούνται πλέον «τεμπέλικα» μέσα στους buffers, και η δομή κληρονομεί τις βέλτιστες I/O επιδόσεις του Buffer Tree.

Σημείωση: Είναι σημαντικό να σημειώσουμε ότι το Buffered Red-Black Tree δεν προκύπτει απλώς προσαρτώντας buffers στους κόμβους ενός τυπικού δυαδικού LLRB δέντρου, καθώς αυτό θα διατηρούσε μικρό βαθμό διακλάδωσης (fan-out 2 ή 3) οδηγώντας σε μη βέλτιστη απόδοση I/O. Αντιθέτως, παρόλο που χρησιμοποιούμε τη λογική του LLRB για τους κανόνες ισορροπίας, είναι απαραίτητη η ομαδοποίηση $\Theta(M/B)$ κόμβων σε «υπερ-κόμβους» (super-nodes), ώστε να επιτευχθεί ο απαιτούμενος υψηλός βαθμός διακλάδωσης που εξασφαλίζει το βέλτιστο επιμερισμένο (amortized) κόστος ενημέρωσης.

7.5 Έξτρα: Ταξινόμηση στην εξωτερική μνήμη

Algorithm 4: External-Multiway-Mergesort (high-level)

Input: Input file F with N items, memory M , block size B
Output: Sorted output file F'

```

// Phase 1 - Run formation
1  $Runs \leftarrow []$ ;
2 while not end-of-file( $F$ ) do
3    $R \leftarrow$  read up to  $M$  items from  $F$ ;
4   sort( $R$ ); // in-memory sort
5   write  $R$  to disk as a new run;
6   append run to  $Runs$ ;

// Phase 2 - Multiway merging
7  $k \leftarrow \lfloor M/B \rfloor$ ; // fan-in
8 while  $|Runs| > 1$  do
9    $NewRuns \leftarrow []$ ;
10  for each group of up to  $k$  runs in  $Runs$  do
11     $R \leftarrow$  merge-group(group); // multi-way merge with one input
12    // buffer per run and one output buffer
13    append  $R$  to  $NewRuns$ ;
14  delete old run files;
15   $Runs \leftarrow NewRuns$ ;
16 rename the remaining run to  $F'$ ;
17 return  $F'$ ;

```

Ανάλυση (I/O complexity).

Ορίζουμε N = αριθμός στοιχείων, M = μέγεθος εσωτερικής μνήμης (σε στοιχεία), B = μέγεθος block, και $k = \Theta(M/B)$.

- *Run formation:* Διαβάζονται και γράφονται όλα τα στοιχεία μία φορά $\Rightarrow \Theta(N/B)$ I/O.
- *Merge passes:* Κάθε πέρασμα διαβάζει και ξαναγράφει όλα τα στοιχεία $\Rightarrow \Theta(N/B)$ I/O ανά πέρασμα. Ο αριθμός περασμάτων είναι $\Theta(\log_k(N/M)) = \Theta(\log_{M/B}(N/B))$.

Άρα το συνολικό κόστος σε I/O είναι

$$\Theta\left(\frac{N}{B} \cdot \log_{M/B} \frac{N}{B}\right).$$

Κάτω φράγμα. Το θεώρημα των Aggarwal–Vitter δίνει το εξής κάτω φράγμα:

$$\Omega\left(\frac{N}{B} \cdot \log_{M/B} \frac{N}{B}\right)$$

για την ταξινόμηση στο εξωτερικό-μνήμη (I/O μοντέλο). Συνεπώς το multiway external mergesort με $k = \Theta(M/B)$ είναι ασυμπτωτικά βέλτιστο όσον αφορά τα I/O.

8 Micro–Macro Τεχνική: Μια Ανασκόπηση

Η τεχνική *micro–macro* είναι ένα γενικό πλαίσιο (framework) για τον σχεδιασμό δυναμικών δομών δεδομένων στο μοντέλο *word RAM*, το οποίο συνδυάζει δομές για εισόδους μικρού μεγέθους (*micro* επίπεδο) με μια ιεραρχική οργάνωση που κλιμακώνει τη λύση στο πλήρες μέγεθος εισόδου (*macro* επίπεδο). Σχετικές ιδέες εμφανίζονται ήδη στο έργο του Mortensen για *fully dynamic orthogonal range reporting* [Mor06], ενώ οι Chan και Tsakalidis τις αναπτύσσουν συστηματικά και τις χρησιμοποιούν ως βασικό σχήμα στα αποτελέσματά τους για *Dynamic Orthogonal Range Searching* (SoCG 2017) και *Dynamic Planar Orthogonal Point Location* (SoCG 2018) [CT17; CT18].

Στόχος της τεχνικής είναι η επίτευξη υπολογαριθμικών χρόνων (π.χ. $O(\log n / \log \log n)$ ή $O(\sqrt{\log n})$), αξιοποιώντας (α) πολύ αποδοτικές *micro*-δομές για σύνολα μεγέθους s (συνήθως $s = \log^{O(1)} n$), συχνά με *packing/bit tricks* και ιδέες *buffering* (βλ. και Section 7.4), και (β) μια *macro*-οργάνωση με αναδρομική αποσύνδεση / δέντρα υψηλού βαθμού και κατάλληλους μετασχηματισμούς που ελέγχουν το κόστος ανά επίπεδο.

Η βασική ιδέα είναι να χωρίσουμε το πρόβλημα σε δύο επίπεδα: (i) *micro-structures* που λύνουν αποδοτικά την ίδια εργασία για μικρό μέγεθος εισόδου s , και (ii) *macro-structures* που συνδέουν πολλές *micro-structures* ώστε να υποστηρίξουν το πλήρες μέγεθος n , μέσω ιεραρχικών αναδρομών, *high-degree* δέντρων και μετασχηματισμών τύπου *van Emde Boas*.

Στα αποτελέσματα των Chan–Tsakalidis η μεθοδολογία εμφανίζεται χαρακτηριστικά ως: πρώτα σχεδιασμός *micro-structures* για “μικρά” input, έπειτα εφαρμογή ενός *van Emde Boas transformation* που ανυψώνει τις *micro-structures* σε μια στενή (*narrow-grid*) σχάρα, και τέλος χρήση μιας *macro-level* δομής (π.χ. *range trees* υψηλού βαθμού) ώστε να καλυφθεί το πλήρες γενικό πρόβλημα [CT17; CT18].

8.1 Rank-space reduction

Σε διαισθητικό επίπεδο, η τεχνική *rank-space reduction* (ή αλλιώς συμπίεση συντεταγμένων - *coordinate compression*) απεικονίζει ένα (μεγάλο) σύνολο αυθαίρετων σημείων σε ένα σύνολο σημείων με μικρότερο εύρος ακέραιων τιμών [GBT84]. Ειδικότερα, αντικαθιστά τις τιμές των συντεταγμένων με τις αντίστοιχες θέσεις τους σε ταξινομημένη σειρά, διατηρώντας τη σχετική διάταξη των στοιχείων, ενώ μειώνεται η “διάσταση” του σύμπαντος τιμών των συντεταγμένων [BCK+08].

Rank space redcution. αυστηρά, έστω $S \subset \mathbb{R}$ ένα πεπερασμένο σύνολο τιμών. Μια *rank-space reduction* αποτελεί μια μονότονη και αμφιμονοσήμαντη απεικόνιση

$$f : S \rightarrow \{1, \dots, m\}, \quad m = |S|,$$

η οποία αντιστοιχεί κάθε στοιχείο στην κατάταξή του (*rank*) στην ταξινομημένη μορφή του S ().

Πολυδιάστατη εκδοχή. Για κάθε σύνολο σημείων $P \subset \mathbb{R}^d$, η διαδικασία εφαρμόζεται ανεξάρτητα σε κάθε συντεταγμένη.

8.1.1 Δυναμικό Σύμπαν: List Labeling

Στη δυναμική περίπτωση η κύρια διαφοροποίηση είναι ότι το σύνολο των σημείων δεν είναι πλέον στατικό αλλά δυναμικό. Επομένως, η κατάταξη των σημείων στην ταξινόμηση ενδέχεται

να αλλάξει. Όταν μια νέα τιμή εισαχθεί ανάμεσα σε δύο υπάρχοντα σημεία, η κατάταξη θα αλλάξει και θα πρέπει να μετακινηθούν οι τιμές των σημείων. Η ενημέρωση με αφελη τρόπο είναι όμως πολύ ακριβή.

Εδώ είναι που χρησιμοποιείται ο αλγόριθμος list labeling.

Monotone list labeling. Συγκεκριμένα, *monotone list labeling* ορίζεται ως το πρόβλημα ανάθεσης ταμπελών (labels) σε ένα δυναμικό σύνολο ολικά διαταγμένων στοιχείων με τέτοιο τρόπο ώστε όταν $x < y$, η ταμπέλα του x να είναι μικρότερη από την ταμπέλα του y . Καθώς εισάγονται σημεία, επιτρέπεται η αλλαγή των ταμπελών [Die82].

Αλγόριθμος. Το παρακάτω είναι γνωστό αποτέλεσμα από τον Paul F. Dietz [Die82]. Μια μονότονη ανάθεση ταμπελών σε n ολικά διατεταγμένα στοιχεία με ταμπέλες στο $[n^{O(1)}]$ μπορεί να συντηρηθεί κατά την εισαγωγή σημείων εκτελώντας $O(n \log n)$ αλλαγές ετικετών.

8.2 Ζέσταμα: Square Root Decomposition

Παρακάτω θα δούμε την απλή εφαρμογή της τετραγωνικής ριζικής αποσύνδεσης (square root decomposition), μιας τεχνικής που χρησιμοποιείται ευρέως για τον δυναμικό διαμερισμό δεδομένων [Fre85].

Ορισμός του προβλήματος. Δεδομένου ενός πίνακα $A[0 \dots n - 1]$, ζητείται η κατασκευή μιας δομής δεδομένων που να υποστηρίζει αποδοτικά τις εξής λειτουργίες:

- `update(i, x)`: θέτει $A[i] = x$,
- `range_sum(l, r)`: επιστρέφει το άθροισμα $\sum_{k=l}^r A[k]$.

Στόχος η βελτίωση του αφελής αλγορίθμου ο οποίος διατρέχει τον πίνακα από το l μέχρι το r και έχει $O(n)$ χρονική πολυπλοκότητα στη χειρότερη περίπτωση.

Λύση. Η βασική ιδέα είναι να τεμαχίσουμε τον πίνακα σε συνεχόμενα blocks, καθένα μεγέθους B (εκτός ίσως από το τελευταίο). Για κάθε block υπολογίζουμε και αποθηκεύουμε το άθροισμα όλων των στοιχείων του. Αυτή η πληροφορία μάς επιτρέπει να απαντούμε σε ερωτήματα αθροίσματος πολύ πιο γρήγορα.

Αλγόριθμος για ερώτημα (query) $[l, r]$:

1. Βρίσκουμε σε ποια blocks ανήκουν τα άκρα l και r .
Αν ανήκουν στο ίδιο block, τότε απλώς σαρώνονται τα στοιχεία από το l έως το r και το άθροισμα επιστρέφεται.
2. Αν το ερώτημα εκτείνεται σε περισσότερα από ένα blocks, τότε:
 - i) Πρώτα σαρώνουμε τα στοιχεία από το l μέχρι το τέλος του block στο οποίο ανήκει (τα οποία είναι το πολύ B στοιχεία).
 - ii) Έπειτα, για όλα τα blocks που βρίσκονται εξ ολοκλήρου ανάμεσα στο block του l και στο block του r , προσθέτουμε απλώς το προϋπολογισμένο άθροισμά τους. Δεν χρειάζεται σάρωση των στοιχείων τους.
 - iii) Τέλος, σαρώνουμε τα στοιχεία στο block του r , ξεκινώντας από την αρχή του block έως το r (αυτά είναι επίσης το πολύ B στοιχεία).

Η ενημέρωση ενός στοιχείου $A[i]$ γίνεται σε $O(1)$: αλλάζουμε την τιμή στον πίνακα και προσαρμόζουμε το αποθηκευμένο άθροισμα του αντίστοιχου block.

Το συνολικό κόστος για ένα (l, r) ερώτημα είναι το πολύ B στοιχεία από την αριστερή πλευρά, το πολύ B από τη δεξιά και περίπου n/B blocks στη μέση, άρα:

$$T(B) = O(B + n/B).$$

Επιλογή B για ελαχιστοποίηση του $B + n/B$

Θεωρούμε τη συνάρτηση:

$$f(B) = B + \frac{n}{B}, \quad B > 0.$$

Υπολογίζουμε την παράγωγο:

$$f'(B) = 1 - \frac{n}{B^2}.$$

Θέτουμε $f'(B) = 0$:

$$1 - \frac{n}{B^2} = 0 \implies B^2 = n \implies B = \sqrt{n}.$$

Επομένως, η επιλογή $B = \sqrt{n}$ ελαχιστοποιεί την συνάρτηση πολυπλοκότητας. Άρα, $T(B) = \Theta(\sqrt{n})$.

8.3 Van Emde Boas Transformation

Ένα κρίσιμο ενδιαμέσο βήμα στη micro–macro φιλοσοφία είναι ένας μετασχηματισμός τύπου *van Emde Boas* (vEB), που “ανεβάζει” μια δομή που δουλεύει για μικρό μέγεθος s σε δομή που δουλεύει για πολύ μεγαλύτερο n , πληρώνοντας συνήθως έναν μικρό (συχνά πολλαπλασιαστικό) παράγοντα $\log \log n$ στο κόστος.

Η κλασική εκδοχή (όπως χρησιμοποιείται/διατυπώνεται σε πλαίσια τύπου Mortensen και επαναχρησιμοποιείται από Chan–Tsakalidis) έχει το εξής σχήμα: υποθέτουμε ότι τα x -coordinates ζουν σε ένα σύνολο X μεγέθους $O(s)$ (στενό πλέγμα $X \times \mathbb{R}$). Τότε, αν έχουμε μια δυναμική δομή για j -sided orthogonal range emptiness πάνω σε s σημεία με χρόνους $U_j(s), Q_j(s)$, μπορούμε να πάρουμε δομή για n σημεία (στο ίδιο $X \times \mathbb{R}$) με κόστος περίπου

$$U(n) = O(U_j(s) \log \log n), \quad Q(n) = O(Q_j(s) \log \log n),$$

δηλαδή “ανεβάζουμε” το μέγεθος από s σε n με επιβάρυνση $\log \log n$ [CT17].

Διαισθητικά, ο vEB-τύπου μηχανισμός είναι μια αναδρομή όπου χωρίζουμε το πρόβλημα σε “slabs” (π.χ. $\sqrt{n}$ οριζόντιες λωρίδες με $\sqrt{n}$ στοιχεία η καθεμία), κρατάμε σε κάθε slab μια περίληψη (summary) μικρού μεγέθους που μπορεί να χειριστεί η micro-structure (π.χ. τα “ακραία” στοιχεία ανά x -τιμή), και αναδρομούμε στα υπόλοιπα. Το βάθος της αναδρομής γίνεται $O(\log \log n)$, εξηγώντας το επιπλέον $\log \log n$ [CT17].

Στο SoCG 2018, οι Chan–Tsakalidis προτείνουν μια παραμετρική παραλλαγή της vEB αναδρομής με παράμετρο b , ώστε να ανταλλάσσουν μέρος της αύξησης στο query-time με αύξηση στο update-time (trade-off), επειδή στο συγκεκριμένο πρόβλημα (sublog query) δεν “χωρά” κανένας $\log \log n$ πολλαπλασιαστικός παράγοντας στα queries [CT18]. Σε επίπεδο report, αυτό το σημείο αξίζει να τονιστεί ως: η vEB μετατροπή δεν είναι απλώς ένα black-box με $\log \log n$ overhead, αλλά μπορεί να παραμετροποιηθεί για λεπτότερο έλεγχο query/update.

8.4 Micro - Macro Τεχνική

Στο micro επίπεδο, ο στόχος είναι μια δομή για μέγεθος s που να έχει “πολύ φθηνές” ενημερώσεις (ιδανικά σχεδόν σταθερές αποσβεσμένα). Η συνήθης στρατηγική είναι να ξεκινήσουμε από μια δομή δεδομένων υλοποιημένη στο internal-memory model και έπειτα να την μετατρέψουμε σε external-memory χρησιμοποιώντας το buffering framework (ώστε οι ενημερώσεις να γίνονται σε batches) και έπειτα να το “επαναυλοποιήσουμε” στην εσωτερική μνήμη μέσω packing, αφού σε word RAM μπορούμε να χωρέσουμε B αντικείμενα σε λίγες λέξεις όταν s είναι πολυλογαριθμικό [CT18]. Αυτό εξηγεί γιατί στις micro-structures εμφανίζονται συχνά ιδέες τύπου buffer trees / buffered search trees ως δομικά συστατικά καθώς και word-ram τεχνικές εκμεταλλευόμενοι το μέγεθος w μιας λέξης.

Στο macro επίπεδο, η ιδέα είναι ότι η δομή η οποία λύνει το πρόβλημα για την πλήρη είσοδο κάνει τα εξής: οργανώνει τα δεδομένα σε ιεραρχία (π.χ. segment tree / range tree, συχνά υψηλού βαθμού) και σε κάθε κόμβο αποθηκεύει “το σωστό” συμπυκνωμένο υποπρόβλημα, το οποίο επιλύεται από micro-structures ή από narrow-grid δομές που προκύπτουν μέσω vEB transformation. Έτσι, το συνολικό κόστος γίνεται:

- (α) κόστος για να κατέβεις/ανέβεις στη macro ιεραρχία (συνήθως $O(\log n)$ ή υπολογαριθμικό μέσω υψηλού βαθμού), και
- (β) κόστος ανά επισκεπτόμενο κόμβο για μικρές δευτερεύουσες δομές (micro ή narrow-grid), με στόχο να είναι αρκετά μικρό ώστε το συνολικό να βγει κάτω από $\log n$.

Συνοπτικά, η micro–macro τεχνική “μεταφέρει” τη δυσκολία: αντί να ζητάμε μια ενιαία δομή με ταυτόχρονα άριστα query/update, φτιάχνουμε micro-δομές που είναι εξαιρετικές σε μικρή κλίμακα, και μετά χρησιμοποιούμε 1–2 προσεκτικά σχεδιασμένους μετασχηματισμούς (vEB/narrow-grid και ένα επιπλέον macro δέντρο) για να τις κλιμακώσουμε στην πλήρη είσοδο [CT17; CT18].

9 Προγραμματιστική Υλοποίηση

Η υλοποίηση της δομής *Priority Queue with Attrition* (PQA) πραγματοποιήθηκε τόσο σε *Python* όσο και σε *C++*, προκειμένου να αξιολογηθούν οι επιδόσεις και οι πρακτικές λεπτομέρειες της λειτουργίας της σε διαφορετικά προγραμματιστικά περιβάλλοντα. Η εργασία περιλαμβάνει την ανάπτυξη της *amortized* παραλλαγής της PQA καθώς και των *worst-case* παραλλαγών, τόσο με την εφαρμογή του *forward cleaning scheme* όσο και του *backwards cleaning scheme*. [Sun89]

Πίνακας 1: Αρχεία υλοποίησης της PQA ανά γλώσσα και παραλλαγή

Γλώσσα	Amortized (Backwards)	Worst-case Forward	Worst-case Backwards
Python	pqa_amortized_backwards.py	pqa_worst_case_forward.py	pqa_worst_case_backwards.py
C++	pqa_amortized_backwards.cpp	pqa_worst_case_forward.cpp	pqa_worst_case_backwards.cpp

Για λόγους καλύτερης διαχείρισης και οργανωμένης παρουσίασης, οι κώδικες υλοποίησης διατηρούνται σε ξεχωριστά αρχεία ανά γλώσσα και ανά παραλλαγή.

10 Ανασκόπηση

Η παρούσα αναφορά και η αντίστοιχη πρακτική άσκηση είχαν ως κύριο στόχο την εκτενή μελέτη και εμβάθυνση σε προηγμένες δομές δεδομένων και τεχνικές υπολογιστικής γεωμετρίας. Κατά τη διάρκεια αυτής της περιόδου, αποκτήθηκαν σημαντικές γνώσεις σε διάφορους τομείς που καλύπτονται στις διαφορετικές ενότητες της αναφοράς. Οι μελέτες επικεντρώθηκαν σε γενικές αρχές διαχείρισης δεδομένων, σχεδίασης και ανάλυσης δομών, οργάνωσης και επεξεργασίας δεδομένων σε διαφορετικά περιβάλλοντα, εξελιγμένες τεχνικές για αποδοτικότητα και αποτελεσματικότητα, καθώς και σε ζητήματα που αφορούν την αξιοποίηση και διαχείριση μνήμης.

Η εμπειρία αυτή προσέφερε πολύτιμες γνώσεις τόσο σε θεωρητικό επίπεδο, με την κατανόηση βασικών αρχών, περιορισμών και αλγοριθμικών trade-offs, όσο και σε πρακτικό επίπεδο, μέσα από την υλοποίηση και αξιολόγηση δομών δεδομένων σε *Python* και *C++*, την αντιμετώπιση προγραμματιστικών προκλήσεων και την ανάπτυξη δεξιοτήτων σε αποδοτικό σχεδιασμό και βελτιστοποίηση.

Ιδιαίτερα, η πρακτική άσκηση αποτέλεσε μια μοναδική ευκαιρία για την εφαρμογή θεωρητικών γνώσεων σε πραγματικά προβλήματα και για την εμβάθυνση σε σύγχρονες τεχνικές της υπολογιστικής γεωμετρίας.

Η ολοκλήρωση αυτής της εργασίας δεν θα ήταν δυνατή χωρίς την πολύτιμη υποστήριξη του Δρ. Κώστα Τσακαλίδη, ο οποίος προσέφερε καθοδήγηση, γνώση και αμέριστη βοήθεια καθ' όλη τη διάρκεια της πρακτικής. Εκφράζονται επίσης θερμές ευχαριστίες στην Ερευνητική Μονάδα *Αρχιμήδης* για την παροχή των υποδομών, της επιστημονικής στήριξης και της ευκαιρίας να συμμετάσχει ο γράφων σε αυτό το πρόγραμμα. Η καθοδήγηση, η εμπειρία και οι πόροι που παρασχέθηκαν συνέβαλαν καθοριστικά στην επιτυχή ολοκλήρωση του έργου και στην απόκτηση των γνώσεων και δεξιοτήτων που περιγράφονται στην αναφορά.

References

- [AL62] Georgy M. Adel'son-Vel'skii and Evgenii M. Landis. "An Algorithm for the Organization of Information". In: *Soviet Mathematics Doklady* 3 (1962).

- English translation of Doklady Akademii Nauk SSSR, pp. 1259–1263. url: <https://zhjwpku.com/assets/pdf/AED2-10-av1-paper.pdf> (visited on 01/21/2026).
- [AV88] Alok Aggarwal and Jeffrey Scott Vitter. “The Input/Output Complexity of Sorting and Related Problems”. In: *Communications of the ACM* 31.9 (1988), pp. 1116–1127. doi: 10.1145/48529.48535.
 - [Ann03] Computational Geometry Mailing List Announcement. *CG models of computation?* CompGeom-announce mailing list post (Dec 7, 2003). 2003.
 - [Arg03] Lars Arge. “The Buffer Tree: A Technique for Designing Batched External Data Structures”. In: *Algorithmica* 37.1 (2003), pp. 1–24. doi: 10.1007/s00453-003-1021-x.
 - [BM72] Rudolf Bayer and Edward M. McCreight. “Organization and Maintenance of Large Ordered Indexes”. In: *Acta Informatica* 1.3 (1972), pp. 173–189. doi: 10.1007/BF00288683.
 - [BDF05] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. “Cache-Oblivious B-Trees”. In: *SIAM Journal on Computing* 35.2 (2005), pp. 341–358. doi: 10.1137/S0097539701389956.
 - [BFJ+15] Michael A. Bender, Martin Farach-Colton, William Jannen, Rob Johnson, Bradley C. Kuszmaul, Donald E. Porter, Jun Yuan, and Yang Zhan. “An Introduction to B_ε-trees and Write-Optimization”. In: *USENIX ;login:* 40.5 (2015), pp. 22–28. url: <https://www.usenix.org/publications/login/oct15/bender>.
 - [BF79] Jon Louis Bentley and Jerome H. Friedman. “Data Structures for Range Searching”. In: *ACM Computing Surveys* 11.4 (1979), pp. 397–409. doi: 10.1145/356789.356797.
 - [BCK+08] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*. 3rd ed. Springer, 2008. isbn: 978-3-540-77973-5. doi: 10.1007/978-3-540-77974-2.
 - [Bla04] Paul E. Black. *pointer machine*. Dictionary of Algorithms and Data Structures, NIST (entry updated 17 Dec 2004). 2004. url: <https://xlinux.nist.gov/dads/HTML/pointermachn.html> (visited on 01/21/2026).
 - [BT11] Gerth Stølting Brodal and Konstantinos Tsakalidis. “Dynamic Planar Range Maxima Queries”. In: *Automata, Languages and Programming – 38th International Colloquium, ICALP 2011, Zurich, Switzerland, July 4–8, 2011, Proceedings, Part I*. Ed. by Luca Aceto, Monika Henzinger, and Jiri Sgall. Vol. 6755. Lecture Notes in Computer Science. Springer, 2011, pp. 256–267. doi: 10.1007/978-3-642-22006-7_22.
 - [CT17] Timothy M. Chan and Konstantinos Tsakalidis. “Dynamic Orthogonal Range Searching on the RAM, Revisited”. In: *33rd International Symposium on Computational Geometry (SoCG 2017)*. 2017, 28:1–28:13. doi: 10.4230/LIPIcs.SocG.2017.28.
 - [CT18] Timothy M. Chan and Konstantinos Tsakalidis. “Dynamic Planar Orthogonal Point Location in Sublogarithmic Time”. In: *34th International Symposium on Computational Geometry (SoCG 2018)*. 2018, 25:1–25:15. doi: 10.4230/LIPIcs.SocG.2018.25.

- [CG86] Bernard Chazelle and Leonidas J. Guibas. “Fractional Cascading: I. A Data Structuring Technique”. In: *Algorithmica* 1.1–4 (1986), pp. 133–162. doi: 10.1007/BF01840440.
- [CR96] Bernard Chazelle and Burton Rosenberg. “Simplex Range Reporting on a Pointer Machine”. In: *Computational Geometry* 5.5 (1996), pp. 237–247. doi: 10.1016/0925-7721(95)00002-X.
- [Com79] Douglas Comer. “The Ubiquitous B-Tree”. In: *ACM Computing Surveys* 11.2 (1979), pp. 121–137. doi: 10.1145/356770.356776.
- [CLR+09] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.
- [Dem+09] Erik D. Demaine et al. *Lecture 10: Sorting III: Sorting Lower Bounds, Counting Sort, Radix Sort (6.006 Fall 2009)*. MIT course lecture notes (PDF). 2009. url: https://courses.csail.mit.edu/6.006/fall09/lecture_notes/lecture10.pdf (visited on 01/21/2026).
- [Dem+12] Erik D. Demaine et al. *6.851: Advanced Data Structures (Lecture 01 scribe notes)*. MIT course scribe notes (Lecture 01, Feb 7, 2012). 2012. url: <https://courses.csail.mit.edu/6.851/spring21/scribe/lec1.pdf> (visited on 01/21/2026).
- [Die82] Paul F. Dietz. “Maintaining Order in a Linked List”. In: *Proceedings of the 14th Annual ACM Symposium on Theory of Computing (STOC)*. 1982, pp. 122–128. doi: 10.1145/800070.802184.
- [DSS+89] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. “Making Data Structures Persistent”. In: *Journal of Computer and System Sciences* 38.1 (1989), pp. 86–124. doi: 10.1016/0022-0000(89)90034-2.
- [Emd77] P. van Emde Boas. “Preserving order in a forest in less than logarithmic time and linear space”. In: *Information Processing Letters* 6.3 (1977), pp. 80–82. doi: 10.1016/0020-0190(77)90031-X.
- [EHM21] Jeff Erickson, Ivor van der Hoog, and Tillmann Miltzow. *Smoothing the Gap Between NP and ER*. Manuscript. Manuscript dated Nov 22, 2021; prior version appeared in FOCS 2020. Nov. 2021. url: <https://jeffe.cs.illinois.edu/pubs/pdf/realram.pdf> (visited on 01/21/2026).
- [FLN+12] Arash Farzan, Alejandro López-Ortiz, Patrick K. Nicholson, and Alejandro Salinger. *Algorithms in the Ultra-Wide Word Model*. Tech. rep. CS-2012-21. University of Waterloo, 2012. url: <https://cs.uwaterloo.ca/research/tr/2012/CS-2012-21.pdf> (visited on 01/21/2026).
- [Fre85] Greg N. Frederickson. “Data Structures for On-Line Updating of Minimum Spanning Trees, with Applications”. In: *SIAM Journal on Computing*. Vol. 14. 4. 1985, pp. 781–798. doi: 10.1137/0214058.
- [Fre60] Edward Fredkin. “Trie Memory”. In: *Communications of the ACM* 3.9 (1960), pp. 490–499. doi: 10.1145/367390.367400.
- [FS89] Michael L. Fredman and Michael E. Saks. “The Cell Probe Complexity of Dynamic Data Structures”. In: *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*. 1989, pp. 345–354. doi: 10.1145/73007.73040.
- [FW93] Michael L. Fredman and Dan E. Willard. “Surpassing the Information Theoretic Bound with Fusion Trees”. In: *Journal of Computer and System Sciences* 47.3 (1993), pp. 424–436. doi: 10.1016/0022-0000(93)90040-4.

- [FW94] Michael L. Fredman and Dan E. Willard. “Trans-Dichotomous Algorithms for Minimum Spanning Trees and Shortest Paths”. In: *Journal of Computer and System Sciences* 48.3 (1994), pp. 533–551. doi: 10.1016/S0022-0000(05)80064-9.
- [FLP+12] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. “Cache-Oblivious Algorithms”. In: *ACM Transactions on Algorithms* 8.1 (2012), 4:1–4:22. doi: 10.1145/2071379.2071383.
- [GBT84] Harold N. Gabow, Jon Louis Bentley, and Robert E. Tarjan. “Scaling and Related Techniques for Geometry Problems”. In: *Proceedings of the 16th Annual ACM Symposium on Theory of Computing (STOC)*. 1984, pp. 135–143. doi: 10.1145/800057.808741.
- [GS78] Leo J. Guibas and Robert Sedgwick. “A Dichromatic Framework for Balanced Trees”. In: *Proceedings of the 19th Annual Symposium on Foundations of Computer Science (FOCS)*. 1978, pp. 8–21. doi: 10.1109/SFCS.1978.3. url: <https://sedgwick.io/wp-content/themes/sedgwick/papers/1978Dichromatic.pdf> (visited on 01/21/2026).
- [Gus97] Dan Gusfield. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997. isbn: 9780521585194. doi: 10.1017/CB09780511574931.
- [Hag98] Torben Hagerup. “Sorting and Searching on the Word RAM”. In: *STACS 98: 15th Annual Symposium on Theoretical Aspects of Computer Science*. Ed. by Michel Morvan, Christoph Meinel, and Daniel Krob. Vol. 1373. Lecture Notes in Computer Science. Springer, 1998, pp. 366–398. doi: 10.1007/BFb0028575.
- [Han04] Yijie Han. “Deterministic sorting in $O(n \log \log n)$ time and linear space”. In: *Journal of Algorithms* 50.1 (2004), pp. 96–105. doi: 10.1016/j.jalgor.2003.09.001.
- [Jac89] Guy Jacobson. *Succinct Static Data Structures*. Tech. rep. CMU-CS-89-112. Technical Report / PhD thesis technical report version. Carnegie Mellon University, Computer Science Department, Jan. 1989. url: <https://gwern.net/doc/cs/algorithm/information/compression/1988-jacobson.pdf> (visited on 01/21/2026).
- [KTT+13] Casper Kejlberg-Rasmussen, Yufei Tao, Konstantinos Tsakalidis, Kostas Tsichlas, and Jeonghun Yoon. “I/O-Efficient Planar Range Skyline and Attrition Priority Queues”. In: *Proceedings of the 32nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2013, New York, NY, USA, June 22–27, 2013*. ACM, 2013, pp. 103–114. doi: 10.1145/2463664.2465225.
- [Knu98] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd ed. Addison-Wesley, 1998. isbn: 0201896850.
- [MS08] Kurt Mehlhorn and Peter Sanders. *Algorithms and Data Structures: The Basic Toolbox*. Springer, 2008. isbn: 978-3-540-77977-3. doi: 10.1007/978-3-540-77978-0.
- [Mil00] Peter Bro Miltersen. *Cell Probe Complexity – a Survey*. Survey manuscript (Feb 15, 2000). 2000. url: <https://www.cs.umd.edu/~gasarch/BLOGPAPERS/cellprobesurvey.pdf> (visited on 01/21/2026).

- [Mor68] Donald R. Morrison. “PATRICIA—Practical Algorithm to Retrieve Information Coded in Alphanumeric”. In: *Journal of the ACM* 15.4 (1968), pp. 514–534. doi: 10.1145/321479.321481.
- [Mor06] Christian Worm Mortensen. “Fully Dynamic Orthogonal Range Reporting on RAM”. In: *SIAM Journal on Computing* 35.6 (2006), pp. 1494–1525. doi: 10.1137/S0097539703436722.
- [MR95] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995. isbn: 0521474655.
- [PD06] Mihai Pătraşcu and Erik D. Demaine. “Logarithmic Lower Bounds in the Cell-Probe Model”. In: *SIAM Journal on Computing* 35.4 (2006), pp. 932–963. doi: 10.1137/S0097539705447256.
- [Pro99] Harald Prokop. “Cache-Oblivious Algorithms”. M.S. thesis. MA thesis. Massachusetts Institute of Technology, 1999.
- [San99] Peter Sanders. “Fast Priority Queues for Cached Memory”. In: *Algorithm Engineering and Experimentation (ALENEX 1999)*. Ed. by Michael T. Goodrich and Catherine C. McGeoch. Vol. 1619. Lecture Notes in Computer Science. Springer, 1999, pp. 316–321. doi: 10.1007/3-540-48518-X_19.
- [Sed08] Robert Sedgewick. *Left-Leaning Red-Black Trees*. Department of Computer Science, Princeton University. Available online at <https://sedgewick.io/wp-content/themes/sedgewick/papers/2008LLRB.pdf>. 2008.
- [SA96] Raimund Seidel and Cecilia R. Aragon. “Randomized Search Trees”. In: *Algorithmica* 16.4/5 (1996), pp. 464–497. doi: 10.1007/BF01940876.
- [Sha78] Michael Ian Shamos. “Computational Geometry”. PhD thesis. Yale University, 1978.
- [Sun89] R. Sundar. “Worst-Case Data Structures for the Priority Queue with Attrition”. In: *Information Processing Letters* 31.2 (1989), pp. 69–75. doi: 10.1016/0020-0190(89)90071-9.
- [Tar85] Robert Endre Tarjan. “Amortized Computational Complexity”. In: *SIAM Journal on Algebraic and Discrete Methods* 6.2 (1985), pp. 306–318. doi: 10.1137/0606031.
- [Vit01] Jeffrey Scott Vitter. “External Memory Algorithms and Data Structures: Dealing with Massive Data”. In: *ACM Computing Surveys* 33.2 (2001), pp. 209–271. doi: 10.1145/384192.384193.
- [Yao81] Andrew Chi-Chih Yao. “Should Tables Be Sorted?” In: *Journal of the ACM* 28.3 (1981), pp. 615–628. doi: 10.1145/322261.322274.